

Prolog Programming

From Syntax to Logic and Problem Solving

*A Complete Beginner-to-Advanced Tutorial
with Examples, Exercises, and Practical Programs*

```
greater(X, Y) :- X > Y.
```

“If X is greater than Y , then $\text{greater}(X, Y)$ is true.”

For university students, beginners in logic programming,
and programmers coming from C, C++, Java, or Python

September 8, 2026

To everyone who ever asked “why won’t it just tell me the answer?” — and then learned to ask Prolog the right question.

This book is a self-contained tutorial written to take a complete beginner from the very first Prolog fact to advanced topics such as constraint logic programming and natural language parsing. All code has been written for SWI-Prolog and follows standard ISO Prolog syntax unless otherwise noted.

Contents

1	Introduction to Prolog	13
1.1	What is Prolog?	13
1.2	What does “Prolog” mean?	13
1.3	History of Prolog	13
1.4	Why Prolog was created	13
1.5	Prolog vs procedural programming	14
1.6	Prolog vs Python/C/C++	14
1.7	Declarative programming	14
1.8	Logic programming	14
1.9	Where Prolog is used	14
1.10	Prolog in Artificial Intelligence	15
1.11	How Prolog thinks about a problem	15
1.12	First Prolog program	15
1.13	Installing a Prolog environment	16
1.14	SWI-Prolog introduction	16
1.15	Running a Prolog program	16
1.16	Interactive Prolog console	16
1.17	First query	16
2	Understanding Prolog Syntax	19
2.1	Basic structure of a Prolog program	19
2.2	Clauses	19
2.3	Facts	19
2.4	Rules	19
2.5	Queries	20
2.6	Predicates	20
2.7	Predicate names	20
2.8	Predicate arity	20
2.9	Terms	20
2.10	Atoms	20
2.11	Numbers	20
2.12	Variables	21
2.13	Anonymous variable	21
2.14	Compound terms	21
2.15	Parentheses	21
2.16	Commas	21
2.17	Semicolon	21
2.18	The :- operator	22
2.19	The period	22
2.20	Comments	22
2.21	Whitespace	22
2.22	Case sensitivity	22
2.23	Lowercase vs uppercase identifiers	22
2.24	Common syntax errors	23
2.25	Complete syntax cheat sheet	23
3	Facts	25
3.1	What is a fact?	25
3.2	Fact syntax	25
3.3	Simple facts	25
3.4	Facts with one argument	25
3.5	Facts with multiple arguments	25

3.6	Representing people	26
3.7	Representing relationships	26
3.8	Facts about objects	26
3.9	Facts about properties	26
3.10	Querying facts	26
3.11	Positive queries	27
3.12	Negative queries	27
3.13	Multiple matching facts	27
3.14	Understanding Prolog's answers	27
3.15	true and false	27
3.16	Using variables in queries	27
3.17	Finding all matching facts	28
3.18	Practical example: family database	28
3.19	Practical example: student database	28
4	Rules	31
4.1	What is a rule?	31
4.2	Rule syntax	31
4.3	Meaning of :-	31
4.4	Head and body of a rule	31
4.5	Conditions	32
4.6	Rules with one condition	32
4.7	Rules with multiple conditions	32
4.8	Logical AND using comma	32
4.9	Logical OR using semicolon	32
4.10	Rules using variables	33
4.11	Rules with multiple arguments	33
4.12	Chained rules	33
4.13	Rules depending on other rules	33
4.14	Understanding rule execution	33
4.15	Examples	34
4.16	Common mistakes	34
5	Variables in Prolog	37
5.1	What is a variable?	37
5.2	Variable naming rules	37
5.3	Why variables start with uppercase letters	37
5.4	Lowercase atoms vs uppercase variables	37
5.5	Single-letter variables	37
5.6	Meaningful variable names	38
5.7	Anonymous variable _	38
5.8	Variable scope	38
5.9	Variables inside facts	38
5.10	Variables inside rules	38
5.11	Variables inside queries	38
5.12	Multiple variables	38
5.13	Same variable appearing multiple times	39
5.14	Different variables	39
5.15	Variable binding	39
5.16	Examples	39
5.17	Common beginner mistakes	39
6	Queries and Goal Solving	41
6.1	What is a query?	41
6.2	Query syntax	41
6.3	Asking yes/no questions	41
6.4	Querying variables	41
6.5	Multiple goals	41
6.6	Query with AND	42

6.7	Query with OR	42
6.8	Getting multiple answers	42
6.9	The semicolon after an answer	42
6.10	Asking for another solution	42
6.11	Understanding true	42
6.12	Understanding false	42
6.13	Query evaluation	42
6.14	Goal satisfaction	43
6.15	Practical query examples	43
7	Unification	45
7.1	What is unification?	45
7.2	Why unification is important	45
7.3	Unification vs assignment	45
7.4	The = operator	45
7.5	Unifying variables	46
7.6	Unifying atoms	46
7.7	Unifying compound terms	46
7.8	Unification with lists	46
7.9	Failed unification	47
7.10	Variable binding	47
7.11	Examples	47
7.12	Step-by-step unification	47
7.13	Common misconceptions	47
8	Pattern Matching	49
8.1	What is pattern matching?	49
8.2	Pattern matching in Prolog	49
8.3	Matching facts	49
8.4	Matching arguments	49
8.5	Matching compound terms	49
8.6	Matching lists	50
8.7	Head-tail matching	50
8.8	Practical examples	50
8.9	Pattern matching vs traditional programming	51
9	Logical Operators and Control	53
9.1	AND	53
9.2	OR	53
9.3	NOT	53
9.4	The comma operator	53
9.5	The semicolon operator	53
9.6	The \+ operator	53
9.7	Operator precedence	54
9.8	Parentheses	54
9.9	Combining logical conditions	54
9.10	Examples	54
9.11	Common logical mistakes	55
10	Arithmetic in Prolog	57
10.1	Numbers in Prolog	57
10.2	Integer values	57
10.3	Floating-point values	57
10.4	Arithmetic expressions	57
10.5	The is operator	57
10.6	Why = and is are different	58
10.7	Addition	58
10.8	Subtraction	58
10.9	Multiplication	58

10.10	Division	58
10.11	Modulo	58
10.12	Arithmetic comparison	59
10.13	>	59
10.14	<	59
10.15	$\geq$	59
10.16	$\leq$	59
10.17	$:=$	59
10.18	$\backslash =$	59
10.19	Operator precedence in arithmetic	60
10.20	Practical examples	60
10.21	Common arithmetic mistakes	60
11	Lists	63
11.1	What is a list?	63
11.2	Empty list	63
11.3	Simple lists	63
11.4	Nested lists	63
11.5	Head and tail	63
11.6	The operator	63
11.7	Matching list structures	64
11.8	Accessing list elements	64
11.9	Checking membership	64
11.10	List length	65
11.11	Appending lists	65
11.12	Reversing lists	65
11.13	Finding maximum/minimum	65
11.14	Summing list elements	65
11.15	Counting elements	66
11.16	Searching a list	66
11.17	Filtering lists	66
11.18	Practical examples	66
12	Recursion	69
12.1	What is recursion?	69
12.2	Why recursion is important in Prolog	69
12.3	Base case	69
12.4	Recursive case	69
12.5	Simple recursive predicates	69
12.6	Factorial	70
12.7	Fibonacci	70
12.8	Counting	71
12.9	List recursion	71
12.10	Sum of a list	71
12.11	Length of a list	71
12.12	Member predicate	71
12.13	Reverse	72
12.14	Recursion with multiple arguments	72
12.15	Understanding recursive execution	72
12.16	Tracing recursion	72
12.17	Common recursion mistakes	73
13	Backtracking	75
13.1	What is backtracking?	75
13.2	Why Prolog uses backtracking	75
13.3	Search space	75
13.4	Multiple solutions	75
13.5	The role of semicolon	75
13.6	Choice points	76

13.7 Automatic backtracking	76
13.8 Backtracking through rules	76
13.9 Backtracking through facts	76
13.10 Backtracking through recursive predicates	76
13.11 Practical examples	76
13.12 Understanding failure-driven execution	77
13.13 Debugging backtracking	77
14 Prolog Execution Model	79
14.1 How Prolog solves a query	79
14.2 Goals	79
14.3 Subgoals	79
14.4 Unification	79
14.5 Resolution	79
14.6 Search tree	79
14.7 Depth-first search	80
14.8 Backtracking	80
14.9 Choice points	80
14.10 Variable bindings	80
14.11 Step-by-step execution example	80
14.12 Trace mode	81
14.13 Using trace	81
14.14 Understanding execution output	81
15 Predicate Design	83
15.1 Designing predicates	83
15.2 Choosing predicate names	83
15.3 Choosing arguments	83
15.4 Predicate arity	83
15.5 Reusable predicates	83
15.6 General predicates	83
15.7 Avoiding unnecessary duplication	84
15.8 Predicate composition	84
15.9 Building a small knowledge base	84
15.10 Good Prolog programming practices	84
16 Negation and Failure	87
16.1 Closed-world assumption	87
16.2 Negation as failure	87
16.3 The $\backslash +$ operator	87
16.4 Difference between false and unknown	87
16.5 Examples	87
16.6 Limitations of negation	88
16.7 Common mistakes	88
16.8 Practical examples	88
17 Compound Terms and Structures	91
17.1 What are compound terms?	91
17.2 Functors	91
17.3 Arguments	91
17.4 Nested structures	91
17.5 Modeling real-world objects	91
17.6 Student records	91
17.7 Employee records	92
17.8 Address structures	92
17.9 Family structures	92
17.10 Pattern matching with structures	92
17.11 Practical examples	92

18 Common Built-in Predicates	95
18.1 member/2	95
18.2 append/3	95
18.3 length/2	95
18.4 reverse/2	96
18.5 sort/2	96
18.6 msort/2	96
18.7 select/3	96
18.8 between/3	96
18.9 atom/1	96
18.10 number/1	97
18.11 integer/1	97
18.12 compound/1	97
18.13 var/1	97
18.14 nonvar/1	97
18.15 ground/1	97
18.16 write/1	98
18.17 writeln/1	98
18.18 nl/0	98
18.19 read/1	98
18.20 fail/0	98
18.21 true/0	99
18.22 false/0	99
19 Input and Output	101
19.1 Reading user input	101
19.2 Writing output	101
19.3 write/1	101
19.4 writeln/1	101
19.5 nl/0	101
19.6 read/1	101
19.7 Formatting output	102
19.8 Interactive programs	102
19.9 Example: simple calculator	102
19.10 Example: student information system	102
20 Conditional Logic	105
20.1 If-then-else	105
20.2 The -> operator	105
20.3 The ; operator	105
20.4 Combining -> and ;	105
20.5 Conditional predicates	106
20.6 Practical examples	106
20.7 Common mistakes	106
21 Finding and Searching Solutions	109
21.1 Search problems in Prolog	109
21.2 Generating candidates	109
21.3 Testing candidates	109
21.4 Generate-and-test	109
21.5 Constraint-style thinking	110
21.6 Simple puzzle solving	110
21.7 Number problems	110
21.8 Logic puzzles	110
21.9 Practical search examples	111

22 Practical Prolog Programs	113
22.1 Family relationship system	113
22.2 Student information system	114
22.3 Employee database	114
22.4 Simple recommendation system	115
22.5 Movie recommendation system	115
22.6 Course prerequisite checker	115
22.7 Medical-style symptom reasoning example	116
22.8 Restaurant recommendation system	116
22.9 Simple expert system	116
22.10 Simple chatbot-style rule system	117
23 Prolog for Artificial Intelligence	119
23.1 Why Prolog is relevant to AI	119
23.2 Knowledge representation	119
23.3 Rule-based systems	119
23.4 Expert systems	119
23.5 Inference	119
23.6 Search	119
23.7 Natural language processing	119
23.8 Symbolic AI	120
23.9 Prolog vs modern machine learning	120
23.10 When Prolog is a good choice	120
24 Debugging Prolog	123
24.1 Common syntax errors	123
24.2 Undefined predicate	123
24.3 Instantiation errors	123
24.4 Type errors	123
24.5 Arithmetic errors	123
24.6 Infinite recursion	123
24.7 Unexpected false	124
24.8 Unexpected multiple answers	124
24.9 Debugging with trace	124
24.10 Reading Prolog error messages	124
24.11 Debugging checklist	124
25 Advanced Prolog Concepts	127
25.1 Dynamic predicates	127
25.2 assert/1	127
25.3 retract/1	127
25.4 retractall/1	127
25.5 cut (!)	128
25.6 Why cut exists	128
25.7 Green cut vs red cut	128
25.8 fail-driven loops	128
25.9 findall/3	128
25.10 bagof/3	129
25.11 setof/3	129
25.12 Higher-order programming	129
25.13 Meta-predicates	129
25.14 call/1	129
25.15 Definite Clause Grammars	129
25.16 Difference lists	130
25.17 Accumulators	130
25.18 Tail recursion	130
25.19 Performance considerations	130

26 Constraint Logic Programming	133
26.1 What is constraint logic programming?	133
26.2 CLP(FD)	133
26.3 Constraints vs normal arithmetic	133
26.4 Domains	133
26.5 ins/2	133
26.6 #=	133
26.7 #\=	134
26.8 #<	134
26.9 #>	134
26.10all_distinct/1	134
26.11labeling/2	134
26.12Example: Sudoku	134
26.13Example: scheduling	135
26.14Example: cryptarithm	135
26.15Practical applications	136
27 Natural Language Processing with Prolog	137
27.1 Introduction to NLP in Prolog	137
27.2 Grammar representation	137
27.3 Definite Clause Grammars	137
27.4 Sentence parsing	137
27.5 Noun phrases	137
27.6 Verb phrases	138
27.7 Simple grammar	138
27.8 Example parser	138
27.9 Querying grammar	138
27.10Practical NLP example	139
28 Problem Solving with Prolog	141
Beginner Problems	141
Intermediate Problems	142
Advanced Problems	143
29 Prolog Programming Patterns	145
29.1 Recursive pattern	145
29.2 Generate-and-test pattern	145
29.3 Accumulator pattern	145
29.4 Search pattern	145
29.5 Knowledge-base pattern	146
29.6 Rule-engine pattern	146
29.7 List-processing pattern	146
29.8 Graph-processing pattern	146
29.9 Constraint-solving pattern	147
30 Final Project: University Academic Advisor Expert System	149
30.1 System design	149
30.2 Knowledge representation	149
30.3 Complete Prolog source code	150
30.4 Detailed explanation	151
30.5 Sample queries	151
30.6 Expected outputs	151
30.7 Execution flow	152
30.8 Extension ideas	152
A Prolog Syntax Cheat Sheet	153
B Common Errors and Fixes	155

C Prolog Practice Problems	157
C.1 Beginner Problems (20)	157
C.2 Intermediate Problems (20)	157
C.3 Advanced Problems (15)	158
C.4 Selected Complete Solutions	159
D Viva / Interview Questions	161
E Quick Reference: Prolog in One Page	163

Chapter 1

Introduction to Prolog

1.1 What is Prolog?

Prolog stands apart from almost every programming language you may have used before. In languages like C, Java, or Python, you write a precise sequence of instructions: do this, then do that, then check this condition, then loop. This style is called **procedural** or **imperative** programming, because you tell the computer the exact procedure to follow.

Prolog works differently. Instead of telling the computer *how* to solve a problem step by step, you describe *what is true* about a problem — you state facts and rules about your world — and then you ask Prolog questions. Prolog uses a built-in reasoning engine to search for answers that logically follow from what you have told it.

Analogy

Think of Prolog as a very literal-minded librarian. You do not tell the librarian which shelf to walk to and in what order to scan books. Instead, you give the librarian a big pile of facts (“this book is on shelf 3”, “shelf 3 is in the science section”) and rules (“a book is a science book if it is on a shelf in the science section”). Then you ask a question: “Is this a science book?” The librarian works out the answer using the facts and rules you provided, not because you told it exactly how to walk there.

1.2 What does “Prolog” mean?

The name **Prolog** comes from the French phrase *programmation en logique*, meaning “programming in logic”. This name reflects the core idea of the language: programs are written as logical statements rather than as sequences of commands.

1.3 History of Prolog

Prolog was created in 1972 by Alain Colmerauer and Philippe Roussel at the University of Aix-Marseille in France, working with the logician Robert Kowalski, who was developing the theoretical foundation of logic programming at the University of Edinburgh. Their goal was to build a language that could process natural language and perform automated reasoning. Prolog quickly became one of the two foundational languages of Artificial Intelligence, alongside Lisp.

1.4 Why Prolog was created

Prolog was created to make it easier to represent knowledge and perform logical reasoning directly in a programming language. Early AI researchers found that writing reasoning systems in procedural languages was clumsy: they had to manually implement search algorithms, backtracking, and pattern matching every single time. Prolog builds all of these mechanisms into the language itself, so the programmer can focus on describing the problem rather than the search process.

1.5 Prolog vs procedural programming

Aspect	Procedural (C, Java, Python)	Prolog
Programming style	Imperative: describe steps	Declarative: describe facts and rules
Control flow	Explicit (loops, if-else)	Implicit (backtracking, unification)
Data	Variables hold changing values	Variables represent unknowns to be solved
Execution	Runs top to bottom	Searches for solutions matching a query
Typical use	General software	AI, reasoning, parsing, search problems

1.6 Prolog vs Python/C/C++

If you come from Python, C, or C++, the biggest adjustment is this: in Prolog, a variable is not a labeled box that stores a value you can overwrite. Instead, a variable is a placeholder that Prolog tries to **bind** to a value that makes your statements true. Once bound within a particular attempt at solving a query, a variable cannot be silently changed like a Python variable can with `x = x + 1`.

Another difference: there are no explicit loops like **for** or **while** in basic Prolog. Repetition is achieved through **recursion** (Chapter 12) and **backtracking** (Chapter 13).

1.7 Declarative programming

A **declarative** program describes *what* should be true, not *how* to compute it. For example, in Prolog you might write:

```
parent(tom, bob).
```

This simply states a fact: “Tom is a parent of Bob.” You are not writing an instruction; you are declaring a piece of knowledge.

1.8 Logic programming

Logic programming is a programming paradigm based on formal logic. A logic program consists of a set of logical statements (facts and rules), and computation happens by asking the system to prove a goal (a query) using those statements. Prolog is the most widely used logic programming language.

1.9 Where Prolog is used

Prolog is used in:

- Expert systems (medical diagnosis, financial advice systems)
- Natural language processing and parsing
- Symbolic AI and knowledge representation
- Automated theorem proving
- Constraint solving and scheduling (via CLP(FD), Chapter 26)
- Educational tools for teaching logic and AI
- Some compiler and static-analysis tools

1.10 Prolog in Artificial Intelligence

Prolog was one of the languages most closely associated with the field of **Symbolic AI** — the branch of AI concerned with representing knowledge as symbols and rules, and performing reasoning over that knowledge, rather than learning statistical patterns from data (as modern machine learning does). We will return to this comparison in Chapter 23.

1.11 How Prolog thinks about a problem

When you give Prolog a query, it tries to prove that the query is true using the facts and rules in its database. It does this by:

1. Looking for a fact or rule whose head matches the query.
2. If it finds a rule, trying to prove each condition in the rule's body, one after another.
3. If any condition fails, backing up (**backtracking**) and trying the next possible match.
4. Continuing until the whole query is proven true, or until every possibility has been tried and failed.

This process is called **resolution**, and we will explore it in full detail in Chapter 14.

1.12 First Prolog program

Let's write and understand your very first Prolog program.

Example: Your First Facts

```
% This is a simple Prolog program
student(rahim).
student(karim).

smart(X) :-
    student(X).
```

Line-by-line explanation:

- `% This is a simple Prolog program` — a comment; Prolog ignores everything after `%` on that line.
- `student(rahim).` — a **fact** stating that `rahim` is a student.
- `student(karim).` — another fact, stating that `karim` is a student.
- `smart(X) :- student(X).` — a **rule** stating: “X is smart if X is a student.” Here `X` is a variable, and `:-` means “if”.

Query examples:

```
?- student(rahim).
true.

?- smart(karim).
true.

?- smart(hasan).
false.
```

1.13 Installing a Prolog environment

The most popular and beginner-friendly Prolog implementation is **SWI-Prolog**, a free, open-source, actively maintained system available on Windows, macOS, and Linux.

- **Windows/macOS:** Download the installer from the official SWI-Prolog website and run it like any normal application.
- **Linux (Debian/Ubuntu):** run `sudo apt-get install swi-prolog` in a terminal.
- **Online:** SWI-Prolog also offers a browser-based sandbox (SWISH) that requires no installation, useful for quick experiments.

1.14 SWI-Prolog introduction

SWI-Prolog provides an interactive shell (called the **top-level** or **toplevel**) where you can type queries directly, as well as the ability to load `.pl` source files containing your facts and rules.

1.15 Running a Prolog program

Suppose you save the program from Section 1.12 in a file called `family.pl`. To load and use it:

```
$ swipl
?- [family].
true.

?- student(rahim).
true.
```

The command `[family].` loads the file `family.pl` into the Prolog session (note: no file extension is written inside the brackets).

1.16 Interactive Prolog console

The Prolog console (toplevel) shows a prompt: `?-`. This prompt is where you type your queries. After you press Enter, Prolog will try to prove your query and respond with `true.`, `false.`, or a set of variable bindings.

1.17 First query

Let's try our very first query interactively, without even loading a file, using Prolog's ability to accept facts as you type them (using **assert**, discussed fully in Chapter 25) — but for now, simply understand that a query always starts with `?-` and ends with a period.

```
?- X = 5.
X = 5.

?- 3 + 4 =:= 7.
true.
```

Common Mistakes

- Forgetting the period (`.`) at the end of a fact, rule, or query — Prolog will wait forever for more input.
- Typing a variable starting with a lowercase letter (e.g. `x`) when you meant a variable — Prolog will treat it as an atom instead (explained in Chapter 2).

- Confusing the interactive prompt `?-` with a shell command prompt.

Chapter Summary

- Prolog is a **declarative, logic programming** language: you state facts and rules, and ask questions.
- It was created in 1972 for natural language processing and AI reasoning.
- Prolog differs fundamentally from procedural languages: no explicit loops, and variables are unknowns to be solved rather than mutable storage.
- SWI-Prolog is the standard, free implementation used throughout this book.
- A Prolog program consists of facts and rules; you interact with it through queries typed at the `?-` prompt.

Exercises

1. In your own words, explain the difference between declarative and procedural programming.
2. Install SWI-Prolog (or open the SWISH online environment) and confirm you can reach the `?-` prompt.
3. Type the fact `fruit(apple).` into a file called `test.pl`, load it, and query `fruit(apple).` What do you see?
4. Why do you think Prolog is popular in Artificial Intelligence research rather than in typical web or mobile app development?
5. Predict what `?- fruit(banana).` will output for the program in Exercise 3, and then test your prediction.

Chapter 2

Understanding Prolog Syntax

This chapter is the most important chapter in the book for building a solid foundation. We will go through every basic syntactic element of Prolog, symbol by symbol, so that when you see any Prolog code later in this book, you know exactly what each piece means.

2.1 Basic structure of a Prolog program

A Prolog program is simply a collection of **clauses**. A clause is either a **fact** or a **rule**. Nothing else is stored permanently in a program file; everything else (queries) is typed interactively or run as a directive.

```
fact1.  
fact2(arg1, arg2).  
  
rule1(X) :-  
    condition1(X),  
    condition2(X).
```

2.2 Clauses

Concept

A **clause** is a single statement in a Prolog program, always ending with a period (.). Every fact is a clause, and every rule is a clause.

2.3 Facts

A **fact** is a clause that states something is unconditionally true.

```
likes(john, pizza).
```

This means: “John likes pizza” is simply true, with no conditions attached. Facts are studied in full detail in Chapter 3.

2.4 Rules

A **rule** is a clause that states something is true *if* certain conditions hold.

```
happy(X) :-  
    likes(X, pizza).
```

This means: “X is happy if X likes pizza.” Rules are studied in full detail in Chapter 4.

2.5 Queries

A **query** (also called a **goal**) is a question you ask Prolog, typed at the `?-` prompt.

```
?- likes(john, pizza).
```

Queries are studied in full detail in Chapter 6.

2.6 Predicates

Concept

A **predicate** is the name given to a fact or rule together with the number of arguments it takes. `likes(john, pizza)` uses the predicate `likes/2` (read: “likes, arity 2”).

2.7 Predicate names

The predicate name is the identifier that appears before the parentheses, such as `likes`, `student`, or `parent`. Predicate names:

- Must start with a lowercase letter.
- Can contain letters, digits, and underscores after the first character.
- Cannot contain spaces.

2.8 Predicate arity

Arity is the number of arguments a predicate takes. `parent(tom, bob)` has arity 2, written `parent/2`. `happy(john)` has arity 1, written `happy/1`. Two predicates with the same name but different arity are considered completely different predicates in Prolog — `likes/2` and `likes/3` do not interfere with each other.

2.9 Terms

Concept

Everything in Prolog — facts, rules, arguments, numbers, lists — is built out of a single unifying concept called a **term**. A term is either an atom, a number, a variable, or a compound term.

2.10 Atoms

An **atom** is a fixed, named constant — a specific, unchanging value. Examples: `john`, `pizza`, `red`, `'Hello World'`. Rules for atoms:

- A simple atom starts with a lowercase letter, e.g. `john`, `apple123`.
- Atoms can also be written entirely inside single quotes to allow spaces, uppercase starts, or special characters, e.g. `'John Smith'`, `'Data Point'`.
- Special symbolic atoms exist too, such as `[]` (the empty list) or operators like `+`.

2.11 Numbers

Prolog supports **integers** (e.g. `5`, `-12`, `0`) and **floating-point numbers** (e.g. `3.14`, `-0.5`). Numbers are also terms, and act like special kinds of atoms in most contexts. Arithmetic with numbers is covered fully in Chapter 10.

2.12 Variables

A **variable** is a placeholder for a value that is not yet known. In Prolog, a variable **must start with an uppercase letter or an underscore**: `X`, `Name`, `Age`, `_Result`. Variables are covered fully in Chapter 5.

Common Mistakes

- Writing `x` instead of `X` — lowercase `x` is treated as an atom, not a variable, and this is one of the single most common beginner errors.
- Assuming a variable can hold different values over time within one proof attempt, like in Python — it cannot; once bound, it stays bound for that attempt.

2.13 Anonymous variable

The underscore `_` on its own is the **anonymous variable**. It means “I don’t care what this value is; don’t bother giving it a name.”

```
has_pet(john, _).
```

This states John has a pet, but we don’t specify which one, and we don’t want Prolog to bother reporting a binding for it.

2.14 Compound terms

A **compound term** consists of a **functor** (a name) followed by one or more arguments in parentheses, such as `parent(tom, bob)` or `point(3, 4)`. Facts and rules are themselves compound terms in their head. Compound terms are the building blocks used to represent structured data (Chapter 17) and lists (Chapter 11).

2.15 Parentheses

Parentheses `( )` enclose the argument list of a compound term (including predicates). `greater(X, Y)` means the functor `greater` applied to the two arguments `X` and `Y`. Parentheses are also used to group logical expressions, exactly as in mathematics.

2.16 Commas

Inside an argument list, a comma `,` separates arguments: `point(3, 4)` has two arguments, `3` and `4`. In the *body* of a rule, a comma between goals means logical **AND**: both goals must succeed.

```
adult(X) :-
    person(X),
    age(X, A),
    A >= 18.
```

Here, the comma between `person(X)`, `age(X, A)`, and `A >= 18` means all three conditions must hold.

2.17 Semicolon

The semicolon `;` in the body of a rule means logical **OR**: at least one of the separated goals must succeed.

```
weekend(Day) :-
    Day = saturday ; Day = sunday.
```

This means: “Day is a weekend if Day is Saturday OR Day is Sunday.”

2.18 The :- operator

The symbol `:-` is read as “if” and is used to define a rule. It separates the **head** (what is being defined) from the **body** (the conditions that must be true). It is technically a built-in operator meaning “is implied by”.

```
head :- body.
```

2.19 The period

Every fact, rule, and query in Prolog must end with a period (`.`) followed by whitespace (a space or newline). The period tells Prolog “this clause is complete.” Forgetting it is the single most common beginner mistake, and it causes Prolog’s console to appear to hang, waiting for the rest of the clause.

2.20 Comments

Prolog supports two comment styles:

```
% A single-line comment starts with a percent sign

/* A block comment
   can span multiple lines */
```

Comments are ignored by Prolog entirely and exist only to help human readers.

2.21 Whitespace

Prolog is generally insensitive to extra spaces, tabs, and blank lines between tokens (except inside atoms or strings). This lets you format your code for readability, such as indenting the body of a rule:

```
sibling(X, Y) :-
    parent(P, X),
    parent(P, Y),
    X \= Y.
```

2.22 Case sensitivity

Prolog is case-sensitive, but with a special rule: the case of the *first letter* of an identifier determines whether it’s an atom or a variable (Sections 2.10 and 2.12). Beyond that first letter, **Name** and **name** are treated as entirely distinct — **Name** is a variable, **name** is an atom, and these are unrelated to each other.

2.23 Lowercase vs uppercase identifiers

Starts with	Interpreted as	Examples
lowercase letter	atom	john, red, parent
uppercase letter	variable	X, Name, Age
underscore	variable (often anonymous)	_, _Result
digit	number	5, 3.14

2.24 Common syntax errors

Common Mistakes

- **Missing period:** `likes(john, pizza)` without a trailing `.` — Prolog will keep waiting for input.
- **Capitalized atom without quotes:** writing `likes(John, pizza)` when you meant the atom `john` — Prolog will treat `John` as a variable instead, which usually changes the meaning of your program silently.
- **Mismatched parentheses:** forgetting a closing `)` causes a syntax error.
- **Using `=` where `is` was needed** for arithmetic (explained fully in Chapter 10).
- **Space before the opening parenthesis of a compound term**, e.g. `likes (john, pizza)` — this can actually cause a parse error in strict ISO Prolog because it changes how the term is parsed; always write the parenthesis immediately after the functor name.

2.25 Complete syntax cheat sheet

Symbol	Meaning
<code>.</code>	End of a clause or query
<code>:-</code>	“if” — separates rule head from body
<code>,</code>	Logical AND (in a body); argument separator (in a term)
<code>;</code>	Logical OR
<code>%</code>	Start of a single-line comment
<code>/* ... */</code>	Block comment
<code>()</code>	Argument list / grouping
<code>[]</code>	List notation (Chapter 11)
<code> </code>	Head/tail separator inside lists
<code>_</code>	Anonymous variable
<code>'...'</code>	Quoted atom (allows spaces/uppercase)

Chapter Summary

- A Prolog program is a set of clauses: facts and rules.
- Every term is an atom, a number, a variable, or a compound term.
- The first letter’s case determines whether an identifier is an atom (lowercase) or a variable (uppercase/underscore).
- `:-` means “if”, `,` means AND, `;` means OR, and every clause ends with a period.

Exercises

1. Identify whether each of the following is an atom, a variable, or a number: `apple`, `Apple`, `42`, `_X`, `'Blue Sky'`.
2. Write a fact stating that “coffee” is a drink, and a rule stating that anyone who likes coffee is “energetic”.
3. Explain, in your own words, the difference between a comma used inside an argument list versus a comma used in a rule body.
4. Find the syntax error in the following code and correct it:

```
likes(john pizza)
```

5. What is the arity of `distance(a, b, 10)`? Write it in `name/arity` form.

Chapter 3

Facts

3.1 What is a fact?

Concept

A **fact** is a statement in Prolog that is unconditionally true. It is the simplest building block of a Prolog program — a direct declaration of knowledge, with no “if” attached.

In real life, a fact might be “The sky is blue” or “Dhaka is the capital of Bangladesh.” We are not deriving these statements from other information — we are simply asserting them as true.

3.2 Fact syntax

```
predicate_name(argument1, argument2, ...).
```

- `predicate_name` — a lowercase identifier describing the relationship or property.
- `(...)` — the argument list, which may be empty for a zero-arity fact like `raining..`
- Each argument is a term (usually an atom, number, or variable, though variables are rare in facts).
- The clause ends with a period.

3.3 Simple facts

Example: A Zero-Argument Fact

```
raining.
```

This simply says “it is raining” — true, unconditionally, with no arguments at all.

3.4 Facts with one argument

```
student(rahim).  
fruit(apple).  
tall(sultan).
```

Each states a single property about a single thing: `rahim` is a student, `apple` is a fruit, `sultan` is tall.

3.5 Facts with multiple arguments

```
parent(tom, bob).  
distance(dhaka, chittagong, 264).
```

Here, `parent(tom, bob)` states a two-way relationship (Tom is a parent of Bob), and `distance(dhaka, chittagong, 264)` states a three-way relationship (the distance between Dhaka and Chittagong is 264, in whatever unit is implied, e.g. kilometers).

3.6 Representing people

```
person(alice).
person(bob).
age(alice, 30).
age(bob, 25).
```

3.7 Representing relationships

```
parent(tom, bob).
parent(bob, ann).
married(tom, mary).
friend(alice, bob).
```

Notice that the *order* of arguments matters and must be interpreted consistently: here, the first argument of `parent/2` is always the parent, and the second is always the child.

3.8 Facts about objects

```
book(prolog_guide).
color(book, blue).
price(prolog_guide, 25).
```

3.9 Facts about properties

```
vegetarian(alice).
vegetarian(bob).
allergic(bob, peanuts).
```

3.10 Querying facts

Once facts are loaded, you ask Prolog whether something is true by typing a query with the same shape as a fact.

Example: Querying

```
student(rahim).
student(karim).
```

```
?- student(rahim).
true.
```

```
?- student(hasan).
false.
```

3.11 Positive queries

A **positive query** asks whether something specific is true, and every argument is filled in with a concrete atom or number:

```
?- parent(tom, bob).  
true.
```

3.12 Negative queries

Prolog has no direct concept of “false facts” — it only knows what you told it. If something is not stated and cannot be derived, querying it simply returns **false**, which we interpret as “not known to be true,” not as an absolute denial. This is called the **closed-world assumption** and is discussed fully in Chapter 16.

```
?- parent(bob, tom).  
false.
```

3.13 Multiple matching facts

If more than one fact matches a query, Prolog can find each one in turn using backtracking (Chapter 13). For example, given:

```
parent(tom, bob).  
parent(tom, liz).
```

```
?- parent(tom, X).  
X = bob ;  
X = liz.
```

Typing a semicolon (;) after the first answer asks Prolog for the next matching fact.

3.14 Understanding Prolog’s answers

Prolog can respond in three main ways:

- **true.** — the query matches exactly, with no variables to report.
- **X = something.** — the query succeeded, and X was bound to a specific value.
- **false.** — no fact or rule makes the query true.

3.15 true and false

It is important to understand that **true** does not mean “this statement is true about the real world” — it means “this statement follows from the facts and rules you provided.” Likewise **false** means “this statement does not follow from what you provided,” not “this is definitely false in reality.”

3.16 Using variables in queries

When you place a variable in a query, you are asking Prolog to find a value that makes the statement true.

```
?- student(X).  
X = rahim ;  
X = karim.
```

3.17 Finding all matching facts

To collect *all* matches at once instead of pressing `;` repeatedly, use the built-in predicate `findall/3` (introduced briefly here, covered fully in Chapter 25):

```
?- findall(X, student(X), Students).  
Students = [rahim, karim].
```

3.18 Practical example: family database

Example: Family Facts

```
parent(tom, bob).  
parent(tom, liz).  
parent(bob, ann).  
parent(bob, pat).  
  
male(tom).  
male(bob).  
female(liz).  
female(ann).  
female(pat).
```

Sample queries and expected output:

```
?- parent(tom, bob).  
true.  
  
?- parent(X, ann).  
X = bob.  
  
?- male(liz).  
false.
```

3.19 Practical example: student database

Example: Student Facts

```
student(rahim, cse).  
student(karim, eee).  
student(nadia, cse).  
  
department(cse, computer_science).  
department(eee, electrical_engineering).
```

```
?- student(rahim, Dept).  
Dept = cse.  
  
?- student(Who, cse).  
Who = rahim ;  
Who = nadia.
```

Common Mistakes

- Forgetting that argument *position* carries meaning: `parent(tom, bob)` is not the same as `parent(bob, tom)`.
- Writing a capitalized atom by mistake, e.g. `student(Rahim)`., which silently turns `Rahim` into a variable rather than the intended atom `rahim`.
- Expecting a query about something unmentioned to return an error rather than simply `false`.

Chapter Summary

- A fact is an unconditional statement of truth, written as `predicate(arguments)`.
- Facts model objects, properties, and relationships.
- Querying a fact checks if it matches something already declared, or searches for values that would make it match when variables are used.
- Multiple facts with the same predicate name represent multiple pieces of the same kind of knowledge.

Exercises

1. Write facts describing three fruits and their colors, e.g. `color(apple, red)`.
2. Using the family database in Section 3.18, write and test a query to find all children of `bob`.
3. What would `?- parent(ann, X)` return, given the facts in Section 3.18? Explain why.
4. Create a small database of at least four books with their authors, then write a query asking for all books by a specific author.
5. Explain why Prolog's `false` does not necessarily mean something is impossible in the real world.

Chapter 4

Rules

4.1 What is a rule?

Concept

A **rule** states that something is true *provided* certain conditions are also true. Rules let you derive new knowledge from existing facts instead of writing it all out by hand.

For example, instead of writing a separate fact for every person who happens to be an adult, you can write one rule: “a person is an adult if their age is 18 or more.”

4.2 Rule syntax

```
head :- body.
```

- **head** — a compound term describing what is being defined (like a fact’s shape).
- **:-** — read as “if”.
- **body** — one or more goals that must be satisfied for the head to be considered true.
- **.** — ends the clause.

4.3 Meaning of :-

:- means logical implication read backwards: “Head is true if Body is true.” It does *not* mean assignment, and it does not mean “then” as in some other languages — it always means “if”.

4.4 Head and body of a rule

```
greater(X, Y) :-  
    X > Y.
```

- **Head:** `greater(X, Y)` — defines the predicate `greater/2`.
- **Body:** `X > Y` — the single condition that must hold.

Full syntax breakdown, symbol by symbol, as requested for every new construct:

- **greater** — the predicate name.
- **X** and **Y** — variables representing the two numbers being compared.
- **()** — the argument list of the head.
- **:-** — “if” / rule definition.
- **>** — the arithmetic “greater than” comparison operator (Chapter 10).
- **.** — end of the clause.

4.5 Conditions

Each goal inside the body of a rule is called a **condition**. A rule may have zero conditions (in which case it behaves like a fact, though this is unusual), one condition, or many conditions joined by commas or semicolons.

4.6 Rules with one condition

Example: Single Condition

```
happy(X) :-  
    rich(X).  
  
rich(bill).
```

```
?- happy(bill).  
true.  
  
?- happy(john).  
false.
```

4.7 Rules with multiple conditions

```
eligible(X) :-  
    age(X, A),  
    A >= 18,  
    citizen(X).
```

This rule says: X is eligible if X has some age A, and A is at least 18, and X is a citizen. **All three** conditions must succeed.

4.8 Logical AND using comma

As introduced in Chapter 2, a comma between goals in a rule body means AND — every comma-separated goal must succeed for the rule to succeed.

4.9 Logical OR using semicolon

A semicolon between goals means OR — at least one side must succeed.

```
discount_eligible(X) :-  
    student(X) ; senior(X).
```

This says X is eligible for a discount if X is a student OR X is a senior. In practice, many Prolog style guides prefer writing this using two separate rules instead, which is often clearer:

```
discount_eligible(X) :- student(X).  
discount_eligible(X) :- senior(X).
```

Both versions behave identically.

4.10 Rules using variables

Variables in a rule connect the head to the body, and connect different goals in the body to each other. The *same* variable name appearing more than once means it must refer to the *same* value everywhere it appears in that clause.

```
grandparent(X, Z) :-  
    parent(X, Y),  
    parent(Y, Z).
```

Here, *Y* links the two *parent* goals together: whoever *Y* is bound to as the child of *X* must also be the parent of *Z*.

4.11 Rules with multiple arguments

```
sibling(X, Y) :-  
    parent(P, X),  
    parent(P, Y),  
    X \= Y.
```

This says *X* and *Y* are siblings if they share a parent *P*, and *X* is not the same person as *Y* (*\=* means “does not unify with”, covered in Chapter 7).

4.12 Chained rules

Rules can call other rules, which can call other rules, building up layers of reasoning.

```
grandparent(X, Z) :-  
    parent(X, Y),  
    parent(Y, Z).  
  
greatgrandparent(X, W) :-  
    grandparent(X, Z),  
    parent(Z, W).
```

4.13 Rules depending on other rules

This chaining is exactly how larger Prolog programs are built: small, single-purpose rules that combine into complex reasoning, much like functions calling other functions in traditional programming — except here, everything is driven by matching and searching for what makes each condition true.

4.14 Understanding rule execution

When Prolog evaluates `grandparent(tom, ann)`, it does the following:

1. Matches the head `grandparent(X, Z)` with `X = tom, Z = ann`.
2. Tries to satisfy `parent(tom, Y)` — finds `Y = bob` (from a fact).
3. With *Y* now bound to *bob*, tries `parent(bob, ann)` — succeeds if this fact exists.
4. If both conditions succeed, the whole rule succeeds, and Prolog reports **true**.

4.15 Examples

Example: A Complete Family Rule System

```
parent(tom, bob).
parent(bob, ann).

grandparent(X, Z) :-
    parent(X, Y),
    parent(Y, Z).
```

```
?- grandparent(tom, ann).
true.

?- grandparent(tom, X).
X = ann.
```

Example: Numeric Rule

```
greater(X, Y) :-
    X > Y.
```

```
?- greater(10, 5).
true.

?- greater(2, 9).
false.
```

4.16 Common mistakes

Common Mistakes

- Forgetting that variables must match consistently: reusing the wrong variable name breaks the intended link between conditions.
- Writing = where a comparison like > was intended (Chapter 7 explains why = means something quite different — unification).
- Confusing ; (OR) with , (AND) — this is one of the most common logical bugs for beginners.
- Forgetting a period at the end of a multi-line rule, since the rule can span several lines before it is finished.

Chapter Summary

- A rule has the form `Head :- Body.` and means “Head is true if Body is true.”
- Commas in the body mean AND; semicolons mean OR.
- Variables shared across goals in a rule enforce that the same value is used everywhere they appear.
- Rules can call other rules, letting you build complex reasoning out of small, reusable pieces.

Exercises

1. Write a rule `teenager(X)` that is true if `X`'s age is between 13 and 19 inclusive (assume a fact `age(X, A)` exists).
2. Using the family facts in Section 4.15, write a rule for `greatgrandparent/2` and test it with an appropriate example.
3. Rewrite the OR-based rule in Section 4.9 using two separate clauses, and confirm both versions give the same answers.
4. Explain, step by step, what happens when Prolog evaluates `sibling(bob, liz)` given suitable `parent` facts.
5. What is wrong with the following rule, and how would you fix it?

```
adult(X) :-  
    age(X, A);  
    A >= 18.
```


Chapter 5

Variables in Prolog

5.1 What is a variable?

Concept

A Prolog **variable** is a placeholder standing for a value that is not yet known but which Prolog will try to determine while proving a goal. Unlike variables in procedural languages, a Prolog variable is not a labeled storage box you can overwrite — it is more like an empty slot that gets filled in (**bound**) once, for the duration of a particular attempt to solve a query.

Analogy

Think of a variable as a blank on a form, like “Name: _____.” Before it is filled in, it could be anything. Once you write a name in that blank, it is fixed for that form. If you need a different name, you fill out a fresh copy of the form (a different attempt at solving the goal) rather than erasing and rewriting the same blank.

5.2 Variable naming rules

- Must begin with an uppercase letter (A–Z) or an underscore (_).
- After the first character, may contain letters, digits, and underscores.
- Examples of valid variables: `X`, `Name`, `Age3`, `_Result`, `Student_Name`.
- Examples of invalid variables: `name` (starts lowercase — this is an atom), `3X` (starts with a digit).

5.3 Why variables start with uppercase letters

This convention exists so that Prolog’s parser can instantly tell atoms and variables apart just by looking at the first character, without needing any extra declaration syntax (like `var` or `let` in other languages). It keeps Prolog’s syntax extremely compact.

5.4 Lowercase atoms vs uppercase variables

	<code>john</code>	<code>John</code>
Type	Atom	Variable
Meaning	A specific, fixed value	An unknown to be solved for
Can it match anything?	Only itself	Yes, it can bind to any term

This is such an important distinction that it deserves repeating: `likes(john, pizza)` is a completely different statement from `likes(John, pizza)`. The first checks a specific fact about someone named John. The second asks Prolog to find *any* value that likes pizza.

5.5 Single-letter variables

Short variable names like `X`, `Y`, `Z` are common in small examples (mirroring mathematical notation), but they can make larger programs hard to read.

5.6 Meaningful variable names

In real programs, prefer descriptive names: `Name`, `Age`, `Salary`, `StudentList`. This is purely a style choice for readability — Prolog treats `X` and `StudentName` identically as far as behavior is concerned.

5.7 Anonymous variable `_`

When you need a variable's position filled but do not care about its value, use the anonymous variable `_`. Every occurrence of `_` in a clause is treated as a *separate, unrelated* variable — unlike named variables, multiple underscores do *not* have to bind to the same value.

```
has_child(X) :-  
    parent(X, _).
```

This says `X` has a child if `X` is a parent of *someone*, without caring who.

5.8 Variable scope

A variable's scope is limited to the single clause (fact or rule) in which it appears. The `X` in one rule has no connection whatsoever to the `X` in a different rule or fact, even if they share the same name.

```
double(X, Y) :-Y is X * 2.  
triple(X, Y) :-Y is X * 3.
```

The `X` in `double/2` and the `X` in `triple/2` are entirely independent variables.

5.9 Variables inside facts

Variables rarely appear inside facts, since facts are meant to state fixed, known information. However, it is syntactically legal:

```
default_color(_, unknown).
```

5.10 Variables inside rules

Variables are essential in rules, since they connect the head to the body and connect different goals to each other, as seen throughout Chapter 4.

5.11 Variables inside queries

Placing a variable in a query asks Prolog to search for a value:

```
?- parent(tom, X).  
X = bob.
```

5.12 Multiple variables

A clause can use as many distinct variables as needed:

```
distance(City1, City2, KM) :-  
    route(City1, City2, KM).
```

5.13 Same variable appearing multiple times

When the same variable name appears more than once in a clause, Prolog enforces that all occurrences must be bound to the *same* value.

```
same_department(X, Y) :-  
    student(X, D),  
    student(Y, D).
```

Here, `D` appears twice, forcing both students to belong to the identical department value.

5.14 Different variables

Two different variable names (like `X` and `Y`) are allowed to end up bound to the same value or different values — nothing forces them apart unless you add an explicit condition like `X \= Y` (Chapter 7).

5.15 Variable binding

Concept

Binding is the act of a variable becoming associated with a specific term during the process of unification (Chapter 7). Once bound within an attempt, a variable behaves like that term everywhere else it occurs in the same clause instance. If Prolog backtracks past the point where the binding was made, the variable becomes unbound again, ready to potentially take a different value on the next attempt.

5.16 Examples

Example: Tracing a Binding

```
color(apple, red).  
color(banana, yellow).
```

```
?- color(apple, C).  
C = red.  
  
?- color(F, yellow).  
F = banana.  
  
?- color(F, C).  
F = apple,  
C = red ;  
F = banana,  
C = yellow.
```

5.17 Common beginner mistakes

Common Mistakes

- Starting a variable with a lowercase letter by accident, turning it into an atom (e.g. writing `name` when `Name` was intended).

- Believing a variable can be reassigned like in Python: `X = 5, X = 6.` does *not* change `X` to 6 — it fails, because `X` is already bound to 5, and Prolog cannot unify 5 with 6 (Chapter 7 explains why).
- Reusing a variable name across unrelated parts of a program and expecting them to be connected, when in fact each clause has its own independent scope.
- Forgetting that every separate underscore `_` is unrelated to every other underscore, even within the same clause.

Chapter Summary

- Variables start with an uppercase letter or underscore and represent unknowns Prolog tries to solve for.
- The anonymous variable `_` is used when a value's position matters but its content does not.
- Variable scope is limited to a single clause.
- The same named variable repeated in a clause must bind to the same value everywhere.
- Once bound, a variable cannot silently change value within the same solution attempt.

Exercises

1. Explain why `likes(mary, Pizza)` behaves very differently from `likes(mary, pizza)`.
2. Write a rule `has_sibling(X)` using an anonymous variable appropriately, given a `sibling/2` predicate.
3. Predict the output of `?- X = 5, X = 6.` and explain why, referring to variable binding.
4. Why does Prolog treat every occurrence of `_` as an unrelated variable, unlike named variables?
5. Given `age(alice, 30).` and `age(bob, 25).`, what would `?- age(alice, X), age(bob, X).` return? Why?

Chapter 6

Queries and Goal Solving

6.1 What is a query?

Concept

A **query** (also called a **goal**) is a question posed to Prolog, asking it to determine whether something is true, and if variables are involved, to find values that make it true. Queries are typed at the interactive `?-` prompt or run automatically as directives inside a file.

6.2 Query syntax

```
?- goal1, goal2, goal3.
```

- `?-` — the query prompt, shown automatically by the Prolog toplevel; you do not type this yourself when typing directly into the prompt (but you do include it when writing a directive in a file).
- `goal1, goal2, ...` — one or more goals, separated by commas (AND) or semicolons (OR).
- `.` — ends the query.

6.3 Asking yes/no questions

When every argument in your query is filled with a concrete value, Prolog simply answers **true** or **false**:

```
?- parent(tom, bob).  
true.
```

6.4 Querying variables

When your query includes variables, Prolog searches for bindings that satisfy the goal, and reports them:

```
?- parent(tom, X).  
X = bob.
```

6.5 Multiple goals

You can combine several goals in one query:

```
?- parent(tom, X), male(X).  
X = bob.
```

This asks: “Find an `X` such that Tom is a parent of `X`, AND `X` is male.”

6.6 Query with AND

As with rule bodies, a comma between goals in a query means logical AND — every goal must succeed, using consistent variable bindings across all of them.

6.7 Query with OR

A semicolon between goals in a query means OR:

```
?- age(bob, 25) ; age(bob, 30).  
true.
```

6.8 Getting multiple answers

If a query can be satisfied in more than one way, Prolog will show the first answer and wait. Pressing `;` (and Enter) asks for another; pressing Enter alone (or typing a period on some systems) stops the search.

```
?- parent(tom, X).  
X = bob ;  
X = liz.
```

6.9 The semicolon after an answer

This use of `;` at the interactive prompt is a special interactive command meaning “give me the next solution via backtracking” — it is closely related to, but not exactly the same context as, the semicolon-as-OR seen inside a program’s rule bodies. Both ultimately rely on the same backtracking machinery (Chapter 13).

6.10 Asking for another solution

Internally, when you request another solution, Prolog backtracks to the most recent choice point (Chapter 13) and tries an alternative fact or rule clause.

6.11 Understanding true

`true` indicates that Prolog successfully proved the query using the facts and rules available. If the query had no variables, that’s the entire answer. If it did have variables, each solution comes with a full set of bindings.

6.12 Understanding false

`false` indicates the query could not be proven — no combination of facts and rules could make it hold. Remember: this reflects your program’s current knowledge, not necessarily objective truth (Section 3.15, and the closed-world assumption in Chapter 16).

6.13 Query evaluation

Prolog evaluates a query left to right: it attempts to satisfy the first goal, and only once that goal succeeds does it move on to try the next goal (using whatever bindings the first goal established). If a later goal fails, Prolog backtracks into the earlier goal to look for an alternative solution there before giving up entirely.

6.14 Goal satisfaction

A goal is **satisfied** when Prolog finds a fact that matches it directly, or a rule whose body can also be fully satisfied. This recursive definition — a goal is satisfied if a matching rule’s body is satisfied — is at the very heart of how Prolog computes anything at all.

6.15 Practical query examples

Example: Combining Facts and Rules in Queries

```
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).

male(tom).
male(bob).
female(liz).
female(ann).

grandparent(X, Z) :-
    parent(X, Y),
    parent(Y, Z).
```

```
?- grandparent(tom, ann).
true.

?- grandparent(tom, Who), female(Who).
Who = ann.

?- parent(tom, Child), male(Child).
Child = bob.
```

Common Mistakes

- Expecting Prolog to automatically show *all* answers at once — by default, the toplevel shows one at a time unless you use `findall/3` or similar (Chapter 25).
- Confusing a query’s semicolon-driven “next answer” request with writing OR logic inside your own program.
- Forgetting that goal order can matter for efficiency (and sometimes correctness) — placing a very general, unconstrained goal first can force Prolog to explore far more possibilities than necessary.

Chapter Summary

- A query asks Prolog to prove a goal, optionally searching for variable bindings that make it true.
- Commas mean AND; semicolons mean OR, in both rule bodies and queries.
- Prolog evaluates goals left to right, backtracking when necessary.
- `true/false` reflect what can be proven from the program’s current knowledge, not absolute truth.

Exercises

1. Using the family database from Section 6.15, write a query to find all grandchildren of `tom`.
2. What is the difference between typing `;` at the Prolog prompt after an answer, versus writing `;` inside a rule's body?
3. Predict and then test: what does `?- parent(tom, X), female(X).` return using Section 6.15's facts?
4. Explain in your own words why Prolog evaluates goals left to right rather than all at once.
5. Write a query using `OR` that checks whether `ann` is either male or female, given the facts in Section 6.15.

Chapter 7

Unification

7.1 What is unification?

Concept

Unification is the process Prolog uses to determine whether two terms can be made identical, by binding variables as necessary. It is the single most fundamental mechanism in all of Prolog — matching facts against queries, matching rule heads against goals, and even performing what looks like “passing arguments” are all really just unification happening behind the scenes.

Analogy

Think of unification like fitting puzzle pieces together. Two fixed shapes either fit or they don't. But if one piece has a flexible, moldable edge (a variable), it can reshape itself to match whatever it is placed against — as long as the rest of the piece still fits.

7.2 Why unification is important

Every single computation step in Prolog — checking a fact, calling a rule, comparing structures — is implemented through unification. Understanding unification deeply is the key to understanding *everything else* in this book, especially recursion (Chapter 12), pattern matching (Chapter 8), and lists (Chapter 11).

7.3 Unification vs assignment

In procedural languages, `x = 5` means “store 5 into the box named x, overwriting whatever was there.” In Prolog, `X = 5` means “try to make X and 5 identical.” If X is currently unbound, this succeeds by binding X to 5. If X is already bound to something else, like 6, this *fails*, because 5 and 6 cannot be made identical.

7.4 The = operator

The `=` operator explicitly triggers unification between its two arguments.

```
?- X = 5.
```

- `X` — a variable, currently unbound.
- `=` — the unification operator.
- `5` — a number (a fixed term).

Since `X` is unbound, Prolog binds it to 5, and the goal succeeds.

7.5 Unifying variables

Two unbound variables can unify with each other, becoming linked so that binding one later binds the other too:

```
?- X = Y.  
X = Y.  
  
?- X = Y, Y = 10.  
X = 10,  
Y = 10.
```

7.6 Unifying atoms

Two atoms unify only if they are exactly identical:

```
?- apple = apple.  
true.  
  
?- apple = orange.  
false.
```

7.7 Unifying compound terms

Two compound terms unify only if they have the same functor, the same arity, and each corresponding pair of arguments unifies:

```
?- point(1, 2) = point(1, 2).  
true.  
  
?- point(1, 2) = point(X, Y).  
X = 1,  
Y = 2.  
  
?- point(1, 2) = point(1, 2, 3).  
false.
```

The last example fails because the arity differs (2 vs 3) — even though the functor name `point` matches.

7.8 Unification with lists

Lists (Chapter 11) are actually compound terms in disguise, so they unify using the same rule: same structure, corresponding parts must unify.

```
?- [1, 2, 3] = [1, 2, 3].  
true.  
  
?- [H|T] = [a, b, c].  
H = a,  
T = [b, c].
```

7.9 Failed unification

Unification fails when two terms simply cannot be made identical, no matter how variables are bound:

```
?- 5 = 6.
false.

?- foo(X) = bar(X).
false.
```

The second example fails because the functors `foo` and `bar` are different — no variable binding can fix a functor mismatch.

7.10 Variable binding

When unification succeeds and involves a variable, that variable becomes **bound** to whatever it was unified with. This binding remains in effect for the rest of that solution attempt, until Prolog backtracks past the point where the binding was created.

7.11 Examples

Example: Unification in a Rule Call

```
greet(Name) :-
    write('Hello, '), write(Name).
```

```
?- greet(mary).
Hello, mary
true.
```

Here, calling `greet(mary)` unifies the head’s variable `Name` with the atom `mary` — this is how “arguments get passed” in Prolog: it’s just unification between the call and the clause head.

7.12 Step-by-step unification

Let’s trace `?- parent(tom, X) = parent(Y, bob).` in full detail:

1. Compare functors: both are `parent` — match.
2. Compare arity: both have 2 arguments — match.
3. Unify first arguments: `tom` with `Y` — since `Y` is unbound, bind `Y = tom`.
4. Unify second arguments: `X` with `bob` — since `X` is unbound, bind `X = bob`.
5. All parts unified successfully, so the whole unification succeeds: `X = bob, Y = tom`.

7.13 Common misconceptions

Common Mistakes

- Believing `=` performs arithmetic: `X = 2 + 3` does *not* compute 5; it binds `X` to the *unevaluated compound term* `2+3`. Use `is` instead for arithmetic (Chapter 10).
- Believing unification can “change” an already-bound variable’s value — it can only succeed if the new value matches what’s already bound, or fail otherwise.

- Forgetting that unifying two terms with different arity always fails, even if the functor name matches.
- Confusing unification (`=`) with equality testing after evaluation (`==`, covered in Chapter 10) — they behave very differently on arithmetic expressions.

Chapter Summary

- Unification tries to make two terms identical by binding variables as needed.
- Atoms unify only with identical atoms; numbers only with equal numbers.
- Compound terms unify only if functor, arity, and every argument unify.
- Unification underlies fact matching, rule calling, and pattern matching throughout Prolog.

Exercises

1. Predict the result of `?- foo(X, 2) = foo(1, Y).` and explain each binding.
2. Why does `?- 5 = 5.0.` fail in standard Prolog even though they represent the same numeric value? (Hint: consider term type, not just numeric value.)
3. Trace step-by-step what happens when unifying `point(X, 3)` with `point(4, Y)`.
4. Explain why `X = 2 + 3` does not produce `X = 5`.
5. Give an example of two compound terms that look similar but fail to unify, and explain why.

Chapter 8

Pattern Matching

8.1 What is pattern matching?

Concept

Pattern matching is the technique of checking whether the *shape* of a piece of data matches a given template, and if so, extracting parts of it. In Prolog, pattern matching is simply unification (Chapter 7) applied to the structure of terms, especially compound terms and lists.

8.2 Pattern matching in Prolog

Because unification already checks structural compatibility and binds variables to matching parts, Prolog gets pattern matching “for free” — you don’t need a separate `switch` or `match` statement like in other languages. You simply write clauses whose heads describe the shapes you want to match.

8.3 Matching facts

When you query `color(apple, C)`, Prolog attempts to unify this pattern against every `color/2` fact until one matches:

```
color(apple, red).  
color(banana, yellow).
```

```
?- color(apple, C).  
C = red.
```

8.4 Matching arguments

You can match on *specific* argument values directly in a clause head, rather than always using a variable and checking afterward:

```
is_red_fruit(apple).  
is_red_fruit(cherry).
```

This works because `apple` in the head of a query like `is_red_fruit(apple)` must unify exactly with the atom `apple` in the fact — a direct pattern match.

8.5 Matching compound terms

Example: Matching Structured Data

```
shape_area(circle(R), A) :-  
    A is 3.14159 * R * R.
```

```
shape_area(rectangle(W, H), A) :-
    A is W * H.
```

Here, the argument pattern `circle(R)` only matches terms actually built with the functor `circle` and one argument, while `rectangle(W, H)` only matches two-argument `rectangle` terms.

```
?- shape_area(circle(2), A).
A = 12.56636.

?- shape_area(rectangle(3, 4), A).
A = 12.
```

8.6 Matching lists

Lists match based on their structure — an empty list only matches `[]`, and a non-empty list can be matched by splitting it into a head and a tail (Section 8.7).

```
?- [] = [].
true.

?- [1,2,3] = [X,Y,Z].
X = 1, Y = 2, Z = 3.
```

8.7 Head-tail matching

The pattern `[Head|Tail]` matches any non-empty list, binding `Head` to the first element and `Tail` to the rest of the list (itself a list).

```
?- [H|T] = [a, b, c].
H = a,
T = [b, c].

?- [H|T] = [x].
H = x,
T = [].
```

This pattern is the foundation of nearly all list-processing recursive predicates (Chapter 11 and 12).

8.8 Practical examples

Example: Using Pattern Matching Instead of if-else

```
first_element([First|_], First).

is_empty_list([]).
```

```
?- first_element([10, 20, 30], X).
X = 10.

?- is_empty_list([]).
true.
```

```
?- is_empty_list([1]).
false.
```

Notice how `first_element/2` uses the anonymous variable `_` to discard the tail, since we only care about the head.

8.9 Pattern matching vs traditional programming

In a language like Python, you might write:

```
def first_element(lst):
    return lst[0]
```

In Prolog, the equivalent is declared as a fact whose head *is* the pattern:

```
first_element([First|_], First).
```

There's no explicit indexing operation — the structure of the argument itself does the work, via unification.

Common Mistakes

- Trying to match a list pattern like `[H|T]` against an atom or a non-list term — this always fails, since an atom has no head/tail structure.
- Forgetting that `[H|T]` requires *at least one* element; it will not match `[]`.
- Writing overlapping clause patterns without considering which one Prolog will try first (Prolog tries clauses in the order they are written, top to bottom).

Chapter Summary

- Pattern matching in Prolog is unification applied to structured data.
- Clause heads act as templates; Prolog matches a call's arguments against these templates automatically.
- `[Head|Tail]` is the key pattern for decomposing lists.
- Prolog needs no separate `switch/match` construct because unification already provides this behavior.

Exercises

1. Write a predicate `second_element(List, X)` that returns the second element of a list using pattern matching.
2. Predict what `?- [H|T] = [].` returns, and explain why.
3. Write two clauses for a predicate `describe/1` that prints “empty list” for `[]` and “has elements” for any non-empty list.
4. Using the `shape_area` example, add a clause for a `square(Side)` shape and test it.
5. Explain why Prolog does not need a separate pattern-matching syntax beyond what facts and rules already provide.

Chapter 9

Logical Operators and Control

9.1 AND

Logical AND, written as a comma `,` between goals, succeeds only if *every* goal joined by it succeeds.

```
can_drive(X) :-  
    age(X, A),  
    A >= 18,  
    has_license(X).
```

9.2 OR

Logical OR, written as a semicolon `;`, succeeds if *at least one* of the joined goals succeeds.

```
is_weekend(Day) :-  
    Day = saturday ; Day = sunday.
```

9.3 NOT

Prolog represents logical NOT using **negation as failure**, written `\+` (backslash plus). `\+ Goal` succeeds if *Goal cannot* be proven — i.e., if *Goal* fails.

```
not_student(X) :-  
    person(X),  
    \+ student(X).
```

This says *X* is a not-a-student if *X* is a person and it is *not provable* that *X* is a student. Negation as failure is examined thoroughly in Chapter 16, including its important limitations.

9.4 The comma operator

Formally, the comma inside a rule body is the built-in operator for conjunction (AND). It has a precise operator precedence in Prolog’s grammar, but for beginners, the practical rule is simple: read a comma as “and then”.

9.5 The semicolon operator

Similarly, the semicolon is the built-in operator for disjunction (OR): read it as “or else”.

9.6 The `\+` operator

`\+` is a prefix operator meaning “it is not provable that”. Symbol breakdown:

- `\` — backslash, part of the operator symbol.

- + — plus sign, part of the operator symbol; together \+ is one indivisible operator token.
- The operator is placed *before* the goal it negates, with a space for clarity: \+ student(X).

9.7 Operator precedence

Just like in arithmetic, where multiplication happens before addition unless parentheses say otherwise, Prolog’s logical operators have a defined precedence: ; (OR) has *lower* precedence than , (AND), meaning AND binds more tightly. So:

```
a, b ; c
```

is understood as (a, b) ; c — “(a AND b) OR c” — not a, (b ; c).

9.8 Parentheses

Use parentheses to make grouping explicit and avoid relying on precedence rules that readers might misremember:

```
valid(X) :-
    (student(X) ; teacher(X)),
    registered(X).
```

Without the parentheses here, the meaning would change to “student(X) OR (teacher(X) AND registered(X))”, which is usually not intended.

9.9 Combining logical conditions

Example: Combining AND, OR, and NOT

```
eligible(X) :-
    (student(X) ; employee(X)),
    \+ blacklisted(X).
```

This says: X is eligible if (X is a student OR X is an employee) AND it is not provable that X is blacklisted.

9.10 Examples

```
?- (1 > 0 ; 1 < 0), 2 > 1.
true.

?- \+ (1 > 5).
true.

?- \+ (1 > 0).
false.
```

9.11 Common logical mistakes

Common Mistakes

- Forgetting that `;` has lower precedence than `,`, leading to unintended groupings when parentheses are omitted.
- Using `\+` on a goal containing unbound variables expecting meaningful bindings afterward — since negation as failure discards any bindings made while testing the goal, variables inside a negated goal generally remain unbound outside it.
- Assuming `\+ Goal` is equivalent to classical logical negation in all cases; it is only reliable when `Goal` is **ground** (contains no unbound variables) — this subtlety is fully explored in Chapter 16.

Chapter Summary

- AND is written with a comma, OR with a semicolon, and NOT with `\+` (negation as failure).
- AND binds more tightly than OR; use parentheses to make intent explicit.
- `\+ Goal` succeeds only when `Goal` cannot be proven true.

Exercises

1. Rewrite `a, b ; c, d` with explicit parentheses showing how Prolog actually groups it.
2. Write a rule `can_enter(X)` that is true if `X` is a member AND `\+ banned(X)`.
3. Predict the result of `?- \+ fail.` and explain why (recall `fail/0` always fails, Chapter 18).
4. Explain why negation as failure can behave unexpectedly if the negated goal contains unbound variables.
5. Combine AND, OR, and NOT into a single rule describing eligibility for a library card, using facts you invent.

Chapter 10

Arithmetic in Prolog

10.1 Numbers in Prolog

Prolog supports two main kinds of numbers: integers and floating-point numbers. Numbers are terms, just like atoms, but they support arithmetic evaluation through special operators.

10.2 Integer values

Integers are whole numbers, positive or negative, with no decimal point: 0, 42, -7. SWI-Prolog supports arbitrarily large integers limited only by memory.

10.3 Floating-point values

Floating-point numbers include a decimal point: 3.14, -0.001, 2.0. Note that 2 and 2.0 are different terms in Prolog — one is an integer, one is a float.

10.4 Arithmetic expressions

An arithmetic expression is a term built from numbers and arithmetic operators, such as `3 + 4 * 2`. On its own, such an expression is *not* evaluated automatically — it is just a compound term with functor `+` and two arguments. To force evaluation, you must use `is` (Section 10.5).

10.5 The `is` operator

```
Result is Expression.
```

- **Result** — a variable (or a number) that will be unified with the computed value.
- **is** — the arithmetic evaluation operator.
- **Expression** — an arithmetic expression built from numbers, variables (already bound to numbers), and arithmetic operators.

```
?- X is 3 + 4.  
X = 7.  
  
?- X is 2 * (3 + 1).  
X = 8.
```

10.6 Why = and is are different

= performs plain unification: `X = 3 + 4` binds `X` to the *unevaluated compound term* `3+4`, not the number 7. `is` *evaluates* the right-hand side arithmetically first, then unifies the result with the left-hand side.

```
?- X = 3 + 4.  
X = 3+4.  
  
?- X is 3 + 4.  
X = 7.
```

This distinction trips up nearly every beginner at least once — remember: `=` builds structure, `is` computes a number.

10.7 Addition

```
?- X is 5 + 3.  
X = 8.
```

10.8 Subtraction

```
?- X is 5 - 3.  
X = 2.
```

10.9 Multiplication

```
?- X is 5 * 3.  
X = 15.
```

10.10 Division

```
?- X is 7 / 2.  
X = 3.5.  
  
?- X is 6 / 2.  
X = 3.
```

In SWI-Prolog, `/` performs true division and may return a float, while integer division is done with `//`:

```
?- X is 7 // 2.  
X = 3.
```

10.11 Modulo

The `mod` operator returns the remainder of integer division:

```
?- X is 7 mod 2.  
X = 1.  
  
?- X is 10 mod 3.  
X = 1.
```

10.12 Arithmetic comparison

Prolog provides a full family of comparison operators for numbers, distinct from = (unification).

10.13 >

Greater than: `?- 5 > 3. → true.`

10.14 <

Less than: `?- 3 < 5. → true.`

10.15 ≥

At least: `?- 5 ≥ 5. → true.`

10.16 ≤

At most (note the unusual symbol order =<, not <=): `?- 3 ≤ 5. → true.`

10.17 ==

Arithmetic equality: evaluates *both* sides and checks if the resulting numbers are equal.

```
?- 2 + 2 == 4.  
true.  
  
?- 2 + 2 = 4.  
false.
```

This second example is a critical illustration: = tries to unify the unevaluated term 2+2 directly with the number 4 — these are structurally different terms, so unification fails. == evaluates both sides first, so it succeeds.

10.18 \=

Arithmetic inequality: true if the evaluated values are different.

```
?- 5 \= 6.  
true.  
  
?- 5 \= 5.  
false.
```

10.19 Operator precedence in arithmetic

Prolog follows standard mathematical precedence: multiplication and division bind more tightly than addition and subtraction, and parentheses override this order, just as in ordinary mathematics.

```
?- X is 2 + 3 * 4.
X = 14.

?- X is (2 + 3) * 4.
X = 20.
```

10.20 Practical examples

Example: Computing a Circle's Area

```
circle_area(Radius, Area) :-
    Area is 3.14159 * Radius * Radius.
```

```
?- circle_area(3, A).
A = 28.27431.
```

Example: Checking Even or Odd

```
is_even(N) :-
    N mod 2 == 0.
```

```
?- is_even(4).
true.

?- is_even(7).
false.
```

10.21 Common arithmetic mistakes

Common Mistakes

- Using `=` instead of `is` when a numeric result is expected.
- Using `==` where unification (`=`) was actually intended, or vice versa.
- Forgetting that all variables in the expression on the right of `is` must already be bound to numbers — `X is Y + 1` fails with an instantiation error if `Y` is unbound.
- Confusing `/` (which may produce a float) with `//` (integer division).

Chapter Summary

- `is` evaluates an arithmetic expression and unifies the result with its left-hand side; `=` only performs structural unification without evaluating.
- Prolog provides `>`, `<`, `>=`, `<=`, `==`, `==\=` for numeric comparisons, all of which evaluate both sides first.

- Standard mathematical precedence and parentheses apply to arithmetic expressions.

Exercises

1. Predict the outputs of `?- X = 2 + 2.` and `?- X is 2 + 2.` and explain the difference.
2. Write a rule `average(A, B, Avg)` that computes the average of two numbers.
3. What does `?- 10 mod 3 == 1.` return? Explain step by step.
4. Write a predicate `max_of(A, B, Max)` using arithmetic comparison and an if-then-else structure (preview of Chapter 20) or two separate clauses.
5. Explain why `X is Y + 1.` produces an error if `Y` has never been bound.

Chapter 11

Lists

11.1 What is a list?

Concept

A **list** in Prolog is an ordered collection of terms. Lists are written using square brackets, with elements separated by commas: `[1, 2, 3]`, `[apple, banana, cherry]`. Internally, every list is actually a compound term built from a special functor (traditionally written `'./2'`), but you rarely need to think about that low-level detail thanks to the convenient bracket notation.

11.2 Empty list

The empty list is written `[]`. It represents a list with zero elements and is the base case for nearly every recursive list predicate (Chapter 12).

11.3 Simple lists

```
Numbers = [1, 2, 3, 4, 5].
Fruits = [apple, banana, cherry].
```

11.4 Nested lists

Lists can contain other lists as elements:

```
Matrix = [[1, 2], [3, 4], [5, 6]].
```

Here, `Matrix` is a list of three elements, and each element is itself a list of two numbers.

11.5 Head and tail

Concept

Every non-empty list can be thought of as having a **head** (its first element) and a **tail** (a list containing all the remaining elements).

For `[1, 2, 3]`: the head is `1`, and the tail is `[2, 3]`.

11.6 The `|` operator

The vertical bar `|` inside list notation separates the head (or heads) from the tail:

```
[Head|Tail]
```

- **Head** — matches the first element.
- **|** — separates head from tail.
- **Tail** — matches everything after the head, itself a list (possibly empty).

You can also split off multiple leading elements: `[A, B|Rest]` matches a list with at least two elements, binding `A` and `B` to the first two, and `Rest` to everything after.

11.7 Matching list structures

```
?- [H|T] = [10, 20, 30].
H = 10,
T = [20, 30].

?- [A, B|Rest] = [1, 2, 3, 4].
A = 1,
B = 2,
Rest = [3, 4].
```

11.8 Accessing list elements

While you can write custom predicates for indexed access, Prolog also provides the built-in `nth0/3` (0-based index) and `nth1/3` (1-based index):

```
?- nth0(0, [a,b,c], X).
X = a.

?- nth1(1, [a,b,c], X).
X = a.
```

11.9 Checking membership

The built-in predicate `member/2` checks (or generates) whether an element belongs to a list. We can also define our own version to understand how it works:

Example: A Custom `member/2`

```
my_member(X, [X|_]).
my_member(X, [_|T]) :-
    my_member(X, T).
```

Line-by-line explanation:

- First clause: `X` is a member of a list if `X` is the head of that list.
- Second clause: `X` is a member of a list if `X` is a member of the tail (recursion — Chapter 12).

```
?- my_member(2, [1,2,3]).
true.

?- my_member(5, [1,2,3]).
false.
```

11.10 List length

The built-in `length/2` computes (or checks) how many elements a list has:

```
?- length([a,b,c], N).  
N = 3.
```

11.11 Appending lists

The built-in `append/3` joins two lists together, and can also be used to split a list in various ways due to Prolog's ability to run predicates "backwards":

```
?- append([1,2], [3,4], X).  
X = [1, 2, 3, 4].  
  
?- append(X, [3,4], [1,2,3,4]).  
X = [1, 2].
```

11.12 Reversing lists

The built-in `reverse/2` reverses a list:

```
?- reverse([1,2,3], R).  
R = [3, 2, 1].
```

11.13 Finding maximum/minimum

Example: Finding the Maximum

```
max_list_custom([X], X).  
max_list_custom([H|T], Max) :-  
    max_list_custom(T, TailMax),  
    (H > TailMax -> Max = H ; Max = TailMax).
```

```
?- max_list_custom([3, 7, 2, 9, 4], Max).  
Max = 9.
```

(SWI-Prolog also provides built-ins `max_list/2` and `min_list/2`.)

11.14 Summing list elements

Example: Summing a List

```
sum_list_custom([], 0).  
sum_list_custom([H|T], Sum) :-  
    sum_list_custom(T, RestSum),  
    Sum is H + RestSum.
```

```
?- sum_list_custom([1,2,3,4], S).
S = 10.
```

11.15 Counting elements

```
count_elements([], 0).
count_elements(_|T, N) :-
    count_elements(T, N1),
    N is N1 + 1.
```

```
?- count_elements([a,b,c,d], N).
N = 4.
```

11.16 Searching a list

```
contains(X, [X|_]) :-!.
contains(X, _|T) :-
    contains(X, T).
```

(The cut ! here is previewed from Chapter 25 to stop unnecessary backtracking once a match is found; you can safely ignore it for now and revisit after reading Chapter 25.)

11.17 Filtering lists

Example: Keeping Only Even Numbers

```
evens_only([], []).
evens_only([H|T], [H|Rest]) :-
    H mod 2 == 0,
    evens_only(T, Rest).
evens_only([H|T], Rest) :-
    H mod 2 \= 0,
    evens_only(T, Rest).
```

```
?- evens_only([1,2,3,4,5,6], X).
X = [2, 4, 6].
```

11.18 Practical examples

Example: Shopping List Total

```
price(bread, 40).
price(milk, 60).
price(eggs, 120).

total_price([], 0).
```

```
total_price([Item|Rest], Total) :-  
    price(Item, Price),  
    total_price(Rest, RestTotal),  
    Total is Price + RestTotal.
```

```
?- total_price([bread, milk, eggs], T).  
T = 220.
```

Common Mistakes

- Forgetting the base case (`[]`) in a recursive list predicate, which causes infinite recursion or failure on the empty list.
- Confusing `[H|T]` (head/tail split) with `[H,T]` (a two-element list) — these are very different patterns.
- Attempting to add an element to a list “in place” as if lists were mutable, like arrays in other languages; Prolog lists are immutable — you always build a *new* list.

Chapter Summary

- Lists are written with square brackets; `[]` is the empty list.
- `[Head|Tail]` splits a list into its first element and the rest.
- Common built-ins include `member/2`, `append/3`, `length/2`, and `reverse/2`.
- Most list-processing predicates are naturally written using recursion, processing one element at a time.

Exercises

1. Write a predicate `last_element(List, Last)` that finds the last element of a list.
2. Using `append/3`, write a query that checks if `[3,4]` is a suffix of `[1,2,3,4]`.
3. Write a predicate `doubles(List, Doubled)` that doubles every number in a list.
4. Trace by hand how `my_member(3, [1,2,3])` succeeds, step by step.
5. Write a predicate that removes all occurrences of a given element from a list.

Chapter 12

Recursion

12.1 What is recursion?

Concept

Recursion occurs when a predicate is defined in terms of itself — a rule whose body calls the very same predicate it is defining, usually with a “smaller” version of the problem, until reaching a simple case that can be answered directly.

Analogy

Imagine a set of Russian nesting dolls. To find out how many dolls are in the set, you could say: “the count is 1 plus however many dolls are inside the next doll.” You keep applying this rule doll after doll until you reach the smallest doll, which contains nothing — at that point, you simply say “0 dolls inside.” Recursion works exactly the same way: break the problem down until you hit the smallest, simplest case.

12.2 Why recursion is important in Prolog

Prolog has no built-in `for` or `while` loop for iterating over data. Repetition is instead achieved through recursion (and through backtracking over multiple facts, Chapter 13). Mastering recursion is therefore essential to writing any non-trivial Prolog program.

12.3 Base case

The **base case** is the simplest version of the problem, which is answered directly without further recursive calls. It is what stops the recursion from continuing forever.

12.4 Recursive case

The **recursive case** handles the general situation by solving a smaller version of the same problem and combining that result with the current step.

12.5 Simple recursive predicates

Example: Counting Down

```
count_down(0) :-  
    write(0), nl.  
count_down(N) :-  
    N > 0,  
    write(N), nl,  
    N1 is N - 1,  
    count_down(N1).
```

Line-by-line explanation:

- First clause (base case): if the argument is exactly 0, print it and stop.
- Second clause (recursive case): if N is greater than 0, print N, compute $N1 = N-1$, and call `count_down` again with the smaller value N1.

```
?- count_down(3).  
3  
2  
1  
0  
true.
```

12.6 Factorial

Example: Factorial

```
factorial(0, 1).  
factorial(N, F) :-  
    N > 0,  
    N1 is N - 1,  
    factorial(N1, F1),  
    F is N * F1.
```

Explanation:

- Base case: the factorial of 0 is 1.
- Recursive case: the factorial of N (for $N > 0$) is N times the factorial of N-1.

```
?- factorial(5, F).  
F = 120.
```

12.7 Fibonacci

Example: Fibonacci Numbers

```
fib(0, 0).  
fib(1, 1).  
fib(N, F) :-  
    N > 1,  
    N1 is N - 1,  
    N2 is N - 2,  
    fib(N1, F1),  
    fib(N2, F2),  
    F is F1 + F2.
```

```
?- fib(6, F).  
F = 8.
```

Note: this simple version recomputes the same values many times and is inefficient for large N; Chapter 25 discusses accumulators, which solve this problem.

12.8 Counting

```
count_up(N, N) :-  
    write(N), nl.  
count_up(N, Max) :-  
    N < Max,  
    write(N), nl,  
    N1 is N + 1,  
    count_up(N1, Max).
```

```
?- count_up(1, 4).  
1  
2  
3  
4  
true.
```

12.9 List recursion

Recursion is the natural way to process every list in Prolog, one element at a time, since lists themselves are recursively structured (a list is either empty, or a head plus a smaller list).

12.10 Sum of a list

```
sum_list_r([], 0).  
sum_list_r([H|T], Sum) :-  
    sum_list_r(T, RestSum),  
    Sum is H + RestSum.
```

```
?- sum_list_r([1,2,3,4], S).  
S = 10.
```

12.11 Length of a list

```
list_length([], 0).  
list_length(_|T, N) :-  
    list_length(T, N1),  
    N is N1 + 1.
```

```
?- list_length([a,b,c], N).  
N = 3.
```

12.12 Member predicate

Revisiting from Chapter 11:

```
my_member(X, [X|_]).  
my_member(X, [_|T]) :-  
    my_member(X, T).
```

12.13 Reverse

Example: Reversing a List Recursively

```
my_reverse([], []).
my_reverse([H|T], Reversed) :-
    my_reverse(T, TailReversed),
    append(TailReversed, [H], Reversed).
```

```
?- my_reverse([1,2,3], R).
R = [3, 2, 1].
```

12.14 Recursion with multiple arguments

```
count_occurrences(_, [], 0).
count_occurrences(X, [X|T], N) :-
    count_occurrences(X, T, N1),
    N is N1 + 1.
count_occurrences(X, [Y|T], N) :-
    X \= Y,
    count_occurrences(X, T, N).
```

```
?- count_occurrences(a, [a,b,a,c,a], N).
N = 3.
```

12.15 Understanding recursive execution

Let's trace `factorial(3, F)` step by step:

1. `factorial(3, F)` calls `factorial(2, F1)`, remembering it needs $F = 3 * F1$ afterward.
2. `factorial(2, F1)` calls `factorial(1, F2)`, remembering $F1 = 2 * F2$.
3. `factorial(1, F2)` calls `factorial(0, F3)`, remembering $F2 = 1 * F3$.
4. `factorial(0, F3)` hits the base case directly: $F3 = 1$.
5. Now the calls “unwind”: $F2 = 1 * 1 = 1$, then $F1 = 2 * 1 = 2$, then $F = 3 * 2 = 6$.

12.16 Tracing recursion

SWI-Prolog's built-in `trace/0` command shows every step of this process live, which is invaluable for debugging recursive predicates (see also Chapter 24).

```
?- trace, factorial(3, F).
```

12.17 Common recursion mistakes

Common Mistakes

- Forgetting the base case entirely, causing infinite recursion until the system runs out of memory (a stack overflow).
- Writing a base case that never gets reached, e.g. only handling `factorial(0, 1)` while decrementing by 2 each time from an odd starting number.
- Placing the recursive call before necessary computations that depend on its result, when the intended logic actually requires computing the result *after* the recursive call returns (as seen in `factorial/2`).
- Confusing the order of arguments between the recursive call and the surrounding computation, leading to reversed or incorrect results.

Chapter Summary

- Recursion defines a predicate in terms of itself, with a base case to stop and a recursive case to make progress.
- Prolog relies on recursion (not loops) for repetition.
- List processing is naturally recursive because lists are themselves recursively structured.
- Tracing execution step by step (mentally or with `trace/0`) is the best way to understand and debug recursive predicates.

Exercises

1. Write a recursive predicate `power(Base, Exp, Result)` computing Base raised to Exp.
2. Trace `fib(4, F)` by hand, showing every recursive call.
3. Write a recursive predicate to count how many even numbers are in a list.
4. Explain what would happen if the base case were removed from `sum_list_r/2`, and why.
5. Write a recursive predicate `list_to_string_length(List, TotalChars)` that sums the lengths of all atoms in a list of atoms (hint: use the built-in `atom_length/2`).

Chapter 13

Backtracking

13.1 What is backtracking?

Concept

Backtracking is Prolog's automatic mechanism for undoing a choice that led to failure, and trying the next available alternative instead. Whenever Prolog has more than one way to satisfy a goal (multiple matching facts, multiple matching rules, or multiple ways of satisfying a sub-goal), it remembers a **choice point** it can return to if things go wrong later.

Analogy

Imagine walking through a maze, and at every junction you pick a path and drop a piece of string behind you. If you hit a dead end, you follow the string back to the last junction and try a different path. Backtracking is exactly this: Prolog explores one possibility, and if it leads to failure, it automatically retraces its steps to the last decision point and tries the next option.

13.2 Why Prolog uses backtracking

Because Prolog programs describe *what* is true rather than a fixed sequence of steps, there can be many possible ways to satisfy a goal. Backtracking is what allows Prolog to systematically explore all of them without you writing any explicit search loop yourself.

13.3 Search space

The **search space** of a query is the full set of possible paths Prolog could explore — every combination of clause choices — while trying to prove that query. Backtracking is how Prolog navigates this space methodically.

13.4 Multiple solutions

```
color(apple, red).  
color(cherry, red).  
color(banana, yellow).
```

```
?- color(Fruit, red).  
Fruit = apple ;  
Fruit = cherry.
```

Prolog finds **apple** first. If you ask for more solutions (pressing `;`), it backtracks and finds **cherry**.

13.5 The role of semicolon

Recall from Chapter 6 that typing `;` at the toplevel after a solution is displayed explicitly asks Prolog to backtrack and search for another solution.

13.6 Choice points

A **choice point** is created whenever Prolog has more than one clause (or fact) that could potentially match the current goal. Prolog remembers where it is in this list of alternatives so it can return there later if needed.

13.7 Automatic backtracking

Backtracking is not something you normally trigger explicitly inside your own rules (aside from a few control constructs like the cut, Chapter 25) — it happens automatically whenever a later goal fails and there is an earlier choice point available.

13.8 Backtracking through rules

Example: Backtracking Across Multiple Rules

```
parent(tom, bob).  
parent(tom, liz).  
parent(bob, ann).  
  
grandparent(X, Z) :-  
    parent(X, Y),  
    parent(Y, Z).
```

When Prolog evaluates `grandparent(tom, Z)`:

1. It tries `parent(tom, Y)`, first matching `Y = bob`.
2. It then tries `parent(bob, Z)`, which succeeds with `Z = ann`.
3. If asked for more solutions, Prolog backtracks into `parent(bob, Z)` — there are no more matches for `bob` — so it backtracks further, into `parent(tom, Y)`, trying the next option `Y = liz`.
4. It then tries `parent(liz, Z)`, which fails (`liz` has no children in our facts).
5. Since there are no more alternatives left anywhere, the search ends.

13.9 Backtracking through facts

As shown above, backtracking through multiple matching facts is the simplest and most direct case: Prolog just tries the next fact with the same predicate name and arity.

13.10 Backtracking through recursive predicates

Recursive predicates create a whole chain of nested choice points, one for each level of recursion, which is why deeply recursive predicates with many matching clauses can sometimes explore very large search spaces.

13.11 Practical examples

Example: Generating All Pairs

```
number(1).  
number(2).
```

```
number(3).

pair(X, Y) :-
    number(X),
    number(Y),
    X \= Y.
```

```
?- pair(X, Y).
X = 1, Y = 2 ;
X = 1, Y = 3 ;
X = 2, Y = 1 ;
X = 2, Y = 3 ;
X = 3, Y = 1 ;
X = 3, Y = 2.
```

This demonstrates backtracking systematically trying every combination — this “generate and test” pattern is explored fully in Chapter 21.

13.12 Understanding failure-driven execution

Prolog’s overall execution strategy can be summarized as: try the first available option; if a later step fails, backtrack and try the next option; repeat until success or until all options are exhausted. This is sometimes called **depth-first search with backtracking** (fully explained in Chapter 14).

13.13 Debugging backtracking

When a program behaves unexpectedly — returning too many or too few solutions — the cause is almost always related to backtracking exploring (or failing to explore) certain paths. Using `trace/0` (Chapter 24) to watch each step, including each backtrack, is the most reliable way to diagnose such issues.

Common Mistakes

- Expecting Prolog to only ever return the first solution — by default it will search for more if asked, which can surprise beginners with duplicate or unexpected extra answers.
- Writing recursive predicates with overlapping clauses that create unwanted extra choice points, leading to duplicate solutions.
- Not realizing that failure in a later goal can silently cause Prolog to retry an *earlier* goal with different bindings, changing behavior in ways that are not obvious just by reading the program top to bottom.

Chapter Summary

- Backtracking automatically retries earlier choice points when a later goal fails.
- Multiple matching facts or rules create choice points that Prolog can revisit.
- Backtracking underlies how Prolog finds multiple solutions to a query.
- The cut operator (Chapter 25) can be used to deliberately prevent backtracking past a certain point when needed.

Exercises

1. Using the facts in Section 13.8, list every solution Prolog would produce for `?- grandparent(tom, Z).` including via backtracking.

2. Explain what a “choice point” is in your own words, using an analogy of your choice.
3. Modify the `pair/2` example to only generate pairs where `X < Y`, removing duplicate reordered pairs.
4. Predict how many solutions `?- color(F, red).` produces given three `red` facts, and explain how backtracking finds each one.
5. Why might a beginner be surprised that a query returns more answers than expected, and how does backtracking explain this?

Chapter 14

Prolog Execution Model

14.1 How Prolog solves a query

Now that you understand facts, rules, unification, and backtracking individually, this chapter ties them together into a complete picture of exactly how Prolog processes a query from start to finish.

14.2 Goals

A **goal** is anything Prolog is trying to prove — the query itself, or any individual condition inside a rule's body. Every goal is ultimately either satisfied (proven true) or fails.

14.3 Subgoals

When a goal matches a rule, the rule's body conditions become **subgoals** that must also be satisfied. Solving the original goal recursively becomes a matter of solving each of its subgoals.

14.4 Unification

As covered in Chapter 7, unification is used at every step to match a goal against a fact or rule head, and this is where variable bindings are created.

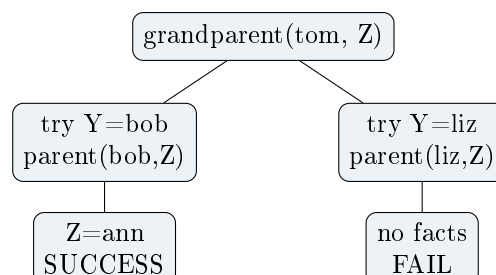
14.5 Resolution

Concept

Resolution is the formal name for the fundamental proof step Prolog performs: given a goal, find a fact or rule head that unifies with it, and (for a rule) replace the goal with the rule's body as new subgoals to solve. Repeating resolution over and over is how Prolog builds a complete proof of a query.

14.6 Search tree

Prolog's process of trying different clauses can be visualized as a **search tree**: the root is the original query, and each branch represents trying a different matching clause. Success corresponds to finding a path from the root to a point where every subgoal has been satisfied.



14.7 Depth-first search

Prolog explores the search tree using **depth-first search**: it commits to the first available branch and follows it all the way down before considering any sibling branches. Only when a branch leads to failure does Prolog backtrack up to try the next sibling. This is why the *order* in which you write your facts and rules can affect which solution Prolog finds first (though not which solutions exist overall).

14.8 Backtracking

As covered fully in Chapter 13, backtracking is how Prolog moves back up the search tree to try an unexplored branch after a failure.

14.9 Choice points

Every node in the search tree with more than one child (more than one applicable clause) represents a choice point, as introduced in Chapter 13.

14.10 Variable bindings

As Prolog moves down the search tree, it accumulates variable bindings from each unification step. When backtracking occurs, all bindings made after the choice point being returned to are undone, so the next branch starts with a clean slate for those variables.

14.11 Step-by-step execution example

Example: Full Trace of a Query

```
likes(mary, wine).
likes(mary, food).
likes(john, wine).

happy(X) :-
    likes(X, wine).
```

Tracing `?- happy(X) :`

1. Goal: `happy(X)`. Matches rule head `happy(X)` — unify, no bindings needed yet for the outer `X` beyond linking it to the rule's internal `X`.
2. New subgoal: `likes(X, wine)`.
3. Try first fact: `likes(mary, wine)` — unifies with `X = mary`.
4. All subgoals satisfied — report `X = mary`.
5. If more solutions are requested: backtrack into `likes(X, wine)`, skip `likes(mary, food)` (does not match “wine”), try `likes(john, wine)` — unifies with `X = john`.
6. Report `X = john`.
7. No more `likes(_, wine)` facts remain — search ends.

```
?- happy(X).
X = mary ;
X = john.
```

14.12 Trace mode

SWI-Prolog provides an interactive debugging tool that shows every resolution step as it happens, called **trace mode**.

14.13 Using trace

```
?- trace.  
?- happy(X).  
  Call: (8) happy(_1234) ?  
  Call: (9) likes(_1234, wine) ?  
  Exit: (9) likes(mary, wine) ?  
  Exit: (8) happy(mary) ?  
X = mary.
```

Type `notrace.` to turn tracing back off.

14.14 Understanding execution output

In trace output, **Call** means Prolog is attempting a goal, **Exit** means the goal succeeded, **Fail** means it failed, and **Redo** means Prolog is backtracking into a previously succeeded goal to look for another solution. Learning to read these four events is the key skill for debugging with trace (Chapter 24).

Chapter Summary

- Prolog solves a query through repeated **resolution**: match a goal to a clause, replace it with the clause body as new subgoals.
- This process can be visualized as a depth-first search through a tree of choices.
- Backtracking moves the search back up the tree after a failure, retrying alternative branches.
- `trace/0` lets you watch this entire process unfold step by step, showing **Call**, **Exit**, **Fail**, and **Redo** events.

Exercises

1. Draw the search tree (by hand or in words) for the query `?- happy(X).` from Section 14.11's facts.
2. What is the difference between **Call** and **Exit** in trace output?
3. Explain, using the concept of depth-first search, why Prolog finds `mary` before `john` in Section 14.11.
4. Use `trace/0` on the `grandparent/2` example from Chapter 13 and describe what you observe.
5. In your own words, explain the relationship between resolution, unification, and backtracking.

Chapter 15

Predicate Design

15.1 Designing predicates

Writing good Prolog is not just about getting a query to return the right answer once — it’s about designing predicates that are reusable, readable, and behave sensibly under backtracking. This chapter focuses on the practical craft of predicate design.

15.2 Choosing predicate names

A predicate name should read naturally in the context `name(Args)`, ideally as a sentence fragment. Compare:

```
parent(tom, bob).      % reads as "tom [is a] parent [of] bob"
is_parent_of(tom, bob).% also fine, more explicit
data1(tom, bob).       % unclear, avoid
```

Favor clarity over brevity, and be consistent across your whole program.

15.3 Choosing arguments

Decide on a consistent **argument order** and stick to it everywhere. A common convention is to place “input” arguments before “output” arguments, e.g. `factorial(N, Result)` takes `N` as input and produces `Result` as output.

15.4 Predicate arity

Keep arity as small as reasonably possible. If a predicate is growing many arguments, consider whether some of them belong together in a compound term (Chapter 17), such as combining separate pieces of data into a single structured record.

15.5 Reusable predicates

Write small predicates that do one thing well, and combine them to build larger behavior — this mirrors the “single responsibility” idea from software engineering. A predicate like `adult(X)` is more reusable than a large predicate that both checks adulthood *and* prints a message *and* looks something up in a file.

15.6 General predicates

Prefer general, parameterized predicates over many near-duplicate specific ones:

```
% Less reusable:
discount_for_student(Price, D) :-D is Price * 0.9.
discount_for_senior(Price, D) :-D is Price * 0.85.

% More reusable:
```

```
discount(student, Price, D) :-D is Price * 0.9.
discount(senior, Price, D) :-D is Price * 0.85.
```

15.7 Avoiding unnecessary duplication

If you find yourself copying and pasting a rule body with only small changes, consider factoring out the common part into its own predicate, exactly as you would extract a shared function in any other language.

15.8 Predicate composition

Complex behavior emerges from composing simpler predicates:

```
can_graduate(Student) :-
    completed_credits(Student, Credits),
    Credits >= 120,
    \+ has_outstanding_fees(Student).
```

15.9 Building a small knowledge base

Example: A Well-Designed Mini Knowledge Base

```
student(rahim, cse, 3.8).
student(karim, eee, 3.2).

honor_roll(Name) :-
    student(Name, _, GPA),
    GPA >= 3.5.
```

```
?- honor_roll(Who).
Who = rahim.
```

15.10 Good Prolog programming practices

- Keep facts and rules for the same predicate grouped together in the file.
- Add a short comment above complex predicates explaining what they compute.
- Choose meaningful variable names in longer rules.
- Test each predicate with several queries, including ones expected to fail, before building further predicates on top of it.
- Prefer several small clauses over one large, deeply nested rule.

Common Mistakes

- Overloading one predicate to do unrelated jobs depending on which arguments are bound, without clear documentation — this makes programs very hard to read.
- Inconsistent argument ordering across similar predicates, which causes frequent bugs when composing them.
- Naming predicates so generically (`check/1`, `process/2`) that their purpose is not clear from the

name alone.

Chapter Summary

- Good predicate design favors clear names, consistent argument ordering, and small, reusable pieces.
- Compose complex behavior from simple predicates rather than writing large monolithic rules.
- Keep related facts and rules organized together and well documented.

Exercises

1. Take the `discount_for_student/discount_for_senior` example and extend it with a third category of your choice using the generalized form.
2. Critique the predicate name `data(X, Y)` — what problems could this cause in a larger program?
3. Design a small knowledge base (at least three predicates) representing a library system, following the good practices listed in Section 15.10.
4. Explain why consistent argument ordering matters when predicates are composed together.
5. Refactor a hypothetical predicate `process_student(Name, Age, GPA, Dept, Result)` into smaller, more focused predicates.

Chapter 16

Negation and Failure

16.1 Closed-world assumption

Concept

Prolog operates under the **closed-world assumption**: anything that cannot be proven true, using the facts and rules currently in the database, is treated as false. This is different from classical logic, where a statement not being proven true does not automatically mean it is false — it might simply be unknown.

For example, if your database has no facts about a particular animal being able to fly, then querying whether it can fly will report **false** — not because Prolog somehow *knows* it cannot fly, but because nothing in the database proves that it can.

16.2 Negation as failure

Prolog’s built-in NOT, `\+`, implements **negation as failure**: `\+ Goal` succeeds exactly when Prolog *fails* to prove `Goal`. This is a pragmatic, computationally efficient approximation of logical negation, but it is not identical to it, as we will see in Section 16.6.

16.3 The `\+` operator

```
not_flying_bird(X) :-  
    bird(X),  
    \+ can_fly(X).
```

Symbol breakdown: `\+` is a prefix operator; when placed before a goal, Prolog tries to prove that goal, and if the attempt fails (no proof found), `\+ Goal` itself succeeds instead.

16.4 Difference between false and unknown

In classical logic, statements can be true, false, or unknown. Prolog only has two outcomes: *provable* and *not provable*. Because of the closed-world assumption, “not provable” is treated identically to “false,” collapsing the “unknown” category into “false.” This works well for many practical programs, but it means you must be careful: absence of information is not the same as a real-world guarantee of falsehood.

16.5 Examples

Example: Negation as Failure in Practice

```
bird(tweety).
bird(penguin1).
can_fly(tweety).
```

```
?- \+ can_fly(penguin1).
true.

?- \+ can_fly(tweety).
false.
```

16.6 Limitations of negation

The most important limitation: negation as failure gives unreliable results if the negated goal contains **unbound variables**. Consider:

```
happy(mary).
happy(john).

not_happy(X) :-
    \+ happy(X).
```

```
?- not_happy(X).
false.
```

This might surprise you: you may have expected Prolog to suggest some *X* that is not happy. But `\+ happy(X)` with an *unbound X* first tries to prove `happy(X)` — which succeeds (with *X* = *mary*) — so the negation fails immediately, *without ever considering any other value of X*. Negation as failure only gives sound results when the goal being negated is **ground** (fully bound, containing no variables) at the time it is evaluated.

16.7 Common mistakes

Common Mistakes

- Using `\+` on a goal with unbound variables, expecting it to search for values that make the goal false — it does not do this; it only checks the single, currently-unbound-variable scenario.
- Assuming `\+ \+ Goal` is the same as `Goal` — it is subtly different: double negation as failure checks provability but always discards any bindings made, leaving variables unbound afterward.
- Forgetting that Prolog's `false` reflects incomplete knowledge, not a guarantee about the real world (Section 16.4).

16.8 Practical examples

Example: Correct Usage: Bind Before Negating

```
person(mary).
person(john).
person(alice).
```

```
happy(mary).  
happy(john).  
  
not_happy(X) :-  
    person(X),  
    \+ happy(X).
```

```
?- not_happy(X).  
X = alice.
```

By first binding `X` via `person(X)` (enumerating specific people) before negating, each candidate is fully ground when `\+ happy(X)` is checked, giving the correct, expected behavior.

Exercises

1. Explain the closed-world assumption in your own words, using a real-life analogy.
2. Given `student(rahim).` and `student(karim).`, what does `?- \+ student(hasan).` return? Why?
3. Rewrite the flawed `not_happy(X)` example from Section 16.6 so it correctly finds all unhappy people, and explain why your fix works.
4. Why is negation as failure considered an approximation of logical negation rather than true logical negation?
5. Give an example (different from the book's) where forgetting to ground a variable before negating it would cause incorrect results.

Chapter 17

Compound Terms and Structures

17.1 What are compound terms?

As introduced in Chapter 2, a **compound term** is a functor applied to one or more arguments, such as `point(3, 4)`. Compound terms let us bundle related pieces of data together into a single structured value, similar to a `struct` in C or a small class instance in Java or Python.

17.2 Functors

The **functor** of a compound term is its name together with its arity, e.g. `point/2` is the functor of `point(3, 4)`. Two compound terms are only considered the “same shape” if they share the same functor.

17.3 Arguments

The values inside the parentheses of a compound term are its **arguments**. Arguments can themselves be any kind of term — atoms, numbers, variables, or even other compound terms.

17.4 Nested structures

Compound terms can be nested to represent richer data:

```
address(street('Main St'), city(dhaka), zip(1207))
```

Here, `address/3` contains three further compound terms as its arguments.

17.5 Modeling real-world objects

Example: A Point in 2D Space

```
point(3, 4).  
  
distance(point(X1,Y1), point(X2,Y2), D) :-  
    D is sqrt((X2-X1)^2 + (Y2-Y1)^2).
```

```
?- distance(point(0,0), point(3,4), D).  
D = 5.0.
```

17.6 Student records

```
student(record(rahim, 21, cse)).
student(record(karim, 22, eee)).

student_name(record(Name, _, _), Name).
```

```
?- student(record(Name, Age, cse)).
Name = rahim, Age = 21.
```

17.7 Employee records

```
employee(emp(nadia, engineering, 75000)).
employee(emp(hasan, sales, 55000)).

high_earner(emp(Name, _, Salary)) :-
    Salary > 60000.
```

17.8 Address structures

```
lives_at(alice, address(house(12), road(5), dhaka)).
```

17.9 Family structures

```
person(name(tom, smith), age(45)).
person(name(ann, smith), age(20)).
```

17.10 Pattern matching with structures

Because compound terms are matched via unification, you can destructure them directly in a clause head, exactly like the list patterns from Chapter 8:

```
full_name(name(First, Last), Full) :-
    atomic_list_concat([First, ' ', Last], Full).
```

```
?- full_name(name(tom, smith), F).
F = 'tom smith'.
```

17.11 Practical examples

Example: A Simple Shape Library

```
area(circle(R), A) :-A is pi * R * R.
area(rectangle(W, H), A) :-A is W * H.
area(triangle(B, H), A) :-A is 0.5 * B * H.
```

```
?- area(triangle(4, 5), A).  
A = 10.0.
```

Common Mistakes

- Confusing a compound term like `point(3,4)` with a list like `[3,4]` — they look similar but have completely different internal representations and matching behavior.
- Forgetting that two compound terms with the same functor name but different arity (e.g. `point/2` vs `point/3`) are entirely unrelated.
- Over-nesting structures so deeply that pattern matching becomes hard to read; consider flattening or renaming fields for clarity.

Chapter Summary

- Compound terms bundle a functor with one or more arguments, letting you represent structured data.
- Nested compound terms can model rich real-world objects like records, addresses, and shapes.
- Pattern matching (via unification) lets you destructure compound terms directly in clause heads.

Exercises

1. Define a compound term structure for representing a book (title, author, year) and write a fact using it.
2. Write a predicate that extracts the year from the book structure in Exercise 1.
3. Explain the difference between `point(3,4)` and `[3,4]` in terms of how Prolog treats them internally.
4. Extend the shape library in Section 17.11 with a `square(Side)` shape.
5. Write a predicate `same_city(P1, P2)` that checks if two `address/3` structures share the same city.

Chapter 18

Common Built-in Predicates

This chapter is a practical reference to the built-in predicates you will use most often. For each, we give its syntax, purpose, an example, expected output, and common mistakes.

18.1 member/2

Purpose: checks whether an element occurs in a list, or generates elements of a list one at a time on backtracking.

```
?- member(2, [1,2,3]).
true.

?- member(X, [a,b,c]).
X = a ;
X = b ;
X = c.
```

Common Mistake

Forgetting `member/2` can be used to *generate* list elements, not just test membership — this is very useful in generate-and-test programs (Chapter 21).

18.2 append/3

Purpose: concatenates two lists, or splits a list into two parts.

```
?- append([1,2],[3,4], X).
X = [1,2,3,4].

?- append(X, Y, [1,2,3]).
X = [], Y = [1,2,3] ;
X = [1], Y = [2,3] ;
X = [1,2], Y = [3] ;
X = [1,2,3], Y = [].
```

18.3 length/2

Purpose: computes the length of a list, or generates a list of a given length filled with unbound variables.

```
?- length([a,b,c], N).
N = 3.

?- length(L, 3).
L = [_A, _B, _C].
```

18.4 reverse/2

Purpose: reverses the order of a list's elements.

```
?- reverse([1,2,3], R).  
R = [3,2,1].
```

18.5 sort/2

Purpose: sorts a list into ascending order and **removes duplicates**.

```
?- sort([3,1,2,1], S).  
S = [1,2,3].
```

Common Mistake

Expecting `sort/2` to preserve duplicate elements — it does not. Use `msort/2` if you need duplicates kept.

18.6 msort/2

Purpose: sorts a list into ascending order **without** removing duplicates.

```
?- msort([3,1,2,1], S).  
S = [1,1,2,3].
```

18.7 select/3

Purpose: removes (or inserts) one occurrence of an element from a list.

```
?- select(2, [1,2,3], Rest).  
Rest = [1,3].
```

18.8 between/3

Purpose: generates (or checks) integers within a range.

```
?- between(1, 5, X).  
X = 1 ;  
X = 2 ;  
X = 3 ;  
X = 4 ;  
X = 5.
```

18.9 atom/1

Purpose: checks whether a term is an atom.

```
?- atom(hello).  
true.  
?- atom(5).  
false.
```

18.10 number/1

Purpose: checks whether a term is a number (integer or float).

```
?- number(5).  
true.  
?- number(hello).  
false.
```

18.11 integer/1

Purpose: checks whether a term is specifically an integer.

```
?- integer(5).  
true.  
?- integer(5.0).  
false.
```

18.12 compound/1

Purpose: checks whether a term is a compound term.

```
?- compound(point(1,2)).  
true.  
?- compound(hello).  
false.
```

18.13 var/1

Purpose: checks whether a term is currently an unbound variable.

```
?- var(X).  
true.  
?- X = 5, var(X).  
false.
```

18.14 nonvar/1

Purpose: the opposite of var/1 — checks that a term is *not* an unbound variable.

```
?- X = 5, nonvar(X).  
true.
```

18.15 ground/1

Purpose: checks that a term contains no unbound variables anywhere (including inside compound terms).

```
?- ground(point(1,2)).
true.
?- ground(point(1,X)).
false.
```

18.16 write/1

Purpose: prints a term to the console without a trailing newline, and without quoting atoms.

```
?- write(hello).
hello
true.
```

18.17 writeln/1

Purpose: like write/1, but automatically adds a newline afterward.

```
?- writeln(hello).
hello
true.
```

18.18 nl/0

Purpose: prints a newline character.

```
?- write(a), nl, write(b).
a
b
true.
```

18.19 read/1

Purpose: reads a term typed by the user from standard input (must end with a period).

```
?- read(X).
|: hello.
X = hello.
```

18.20 fail/0

Purpose: always fails immediately; used to force backtracking, often inside fail-driven loops (Chapter 25).

```
?- fail.
false.
```

18.21 true/0

Purpose: always succeeds immediately, with no effect; useful as a placeholder or a trivial rule body.

```
?- true.  
true.
```

18.22 false/0

Purpose: always fails immediately (an alternative, more explicit spelling of `fail/0` in ISO-compliant systems, including SWI-Prolog).

```
?- false.  
false.
```

Chapter Summary

- Built-in predicates like `member/2`, `append/3`, `length/2`, `reverse/2`, `sort/2` handle the vast majority of list-processing needs without you writing your own recursive versions.
- Type-checking predicates (`atom/1`, `number/1`, `var/1`, etc.) let you write predicates that behave differently depending on what kind of term they receive.
- `write/1`, `writeln/1`, and `nl/0` are the basic tools for producing console output (Chapter 19 covers I/O in depth).

Exercises

1. Use `between/3` to write a query that prints all even numbers from 2 to 10.
2. Explain the difference between `sort/2` and `msort/2` with an example list containing duplicates.
3. Write a predicate that uses `var/1` to check whether an argument was left unbound before proceeding.
4. Use `select/3` to remove the number 5 from a list of your choice and display the result.
5. Explain why `ground(point(1,X))` returns false while `ground(point(1,2))` returns true.

Chapter 19

Input and Output

19.1 Reading user input

Prolog can read data typed by the user during a running program using `read/1`, which reads a full Prolog term ended by a period.

19.2 Writing output

Output is produced using `write/1`, `writeln/1`, `nl/0`, and the more powerful `format/2` predicate (introduced in Section 19.7).

19.3 `write/1`

```
?- write('Hello, Prolog!').  
Hello, Prolog!  
true.
```

19.4 `writeln/1`

```
?- writeln('Hello, Prolog!').  
Hello, Prolog!  
true.
```

`writeln/1` is simply `write/1` followed automatically by `nl/0`.

19.5 `nl/0`

Prints a newline. Useful for formatting multi-line output manually:

```
greet :-  
    write('Hello'), nl,  
    write('Welcome to Prolog'), nl.
```

19.6 `read/1`

```
ask_name :-  
    write('Enter your name: '),  
    read(Name),  
    format("Hello, ~w!~n", [Name]).
```

Note: when typing input for `read/1`, you must end it with a period, e.g. typing `rahim.` and pressing Enter.

19.7 Formatting output

SWI-Prolog's `format/2` predicate provides much more control than `write/1`, using directive codes:

- `~w` — insert the next argument, written with `write/1` style.
- `~a` — insert the next argument as an atom.
- `~d` — insert the next argument as an integer.
- `~n` — insert a newline.

```
?- format("Name: ~w, Age: ~d~n", [rahim, 21]).
Name: rahim, Age: 21
true.
```

19.8 Interactive programs

By combining `read/1`, `write/1`/`format/2`, and recursion, you can build simple interactive command-line programs, as shown in the next two examples.

19.9 Example: simple calculator

Example: A Tiny Interactive Calculator

```
calculator :-
    write('Enter first number: '), read(A),
    write('Enter operator (+,-,*,/): '), read(Op),
    write('Enter second number: '), read(B),
    compute(Op, A, B, Result),
    format("Result: ~w~n", [Result]).
```

```
compute(+, A, B, R) :-R is A + B.
compute(-, A, B, R) :-R is A - B.
compute(*, A, B, R) :-R is A * B.
compute(/, A, B, R) :-R is A / B.
```

```
?- calculator.
Enter first number: 10.
Enter operator (+,-,*,/): +.
Enter second number: 5.
Result: 15
true.
```

19.10 Example: student information system

Example: A Tiny Interactive Lookup Program

```
student(rahim, cse, 3.8).
student(karim, eee, 3.2).
```

```
lookup_student :-
```

```
write('Enter student name: '), read(Name),  
( student(Name, Dept, GPA)  
-> format("Department: ~w, GPA: ~w~n", [Dept, GPA])  
; writeln('Student not found.')
```

```
?- lookup_student.  
Enter student name: rahim.  
Department: cse, GPA: 3.8  
true.
```

(The `-> / ;` construct here is if-then-else, covered fully in Chapter 20.)

Common Mistakes

- Forgetting to end typed input to `read/1` with a period — Prolog will wait indefinitely.
- Confusing `write/1` (no automatic newline) with `writeln/1` (automatic newline).
- Using `format/2` directive codes that do not match the number or type of arguments provided in the list.

Chapter Summary

- Basic output uses `write/1`, `writeln/1`, and `nl/0`; more advanced formatted output uses `format/2`.
- Input is read with `read/1`, which expects a complete Prolog term ending in a period.
- Combining I/O predicates with recursion and conditionals allows building simple interactive programs.

Exercises

1. Modify the calculator example to also handle modulo (`mod`).
2. Write a program that asks the user for their age and prints whether they are a minor or an adult.
3. Use `format/2` to print a small formatted table of three students and their GPAs.
4. Explain why forgetting the trailing period when typing input to `read/1` causes the program to appear to hang.
5. Extend the student lookup example to also print the student's name in the output message.

Chapter 20

Conditional Logic

20.1 If-then-else

Concept

Prolog provides an explicit **if-then-else** construct, built from the `->` (if-then) operator combined with `;` (else). This gives you a way to express conditional logic directly, similar to `if/else` in procedural languages, though it is used more sparingly in idiomatic Prolog (which often prefers multiple clauses instead).

20.2 The `->` operator

```
Condition ->ThenGoal
```

`Condition -> ThenGoal` means: if `Condition` succeeds (at least once), commit to that and then try `ThenGoal`; if `Condition` fails, the whole `->` expression fails (without an `;` else-branch, there is nowhere else to go).

20.3 The `;` operator

When combined with `;`, you get full if-then-else:

```
( Condition ->ThenGoal ;ElseGoal )
```

- `Condition` — the test.
- `->` — “then”, committing to the first solution of `Condition` if it succeeds.
- `ThenGoal` — executed if `Condition` succeeded.
- `;` — “else”.
- `ElseGoal` — executed if `Condition` failed.
- Outer parentheses are required to group the whole if-then-else correctly, due to operator precedence (Chapter 9).

20.4 Combining `->` and `;`

Example: Basic If-Then-Else

```
classify(N, Result) :-  
    ( N >= 0  
    -> Result = non_negative  
    ; Result = negative  
    ).
```

```
?- classify(5, R).
R = non_negative.

?- classify(-3, R).
R = negative.
```

20.5 Conditional predicates

An important behavioral difference from ordinary rules: once the **Condition** in an if-then-else succeeds, Prolog **commits** to it — it will not backtrack into **Condition** to look for alternative solutions, even if **ThenGoal** later fails. This makes if-then-else behave somewhat like the cut (Chapter 25) with respect to the condition.

20.6 Practical examples

Example: Grading System

```
grade(Score, Grade) :-
    ( Score >= 80 ->Grade = 'A'
    ; Score >= 60 ->Grade = 'B'
    ; Score >= 40 ->Grade = 'C'
    ; Grade = 'F'
    ).
```

This chains several if-then-else expressions together to express multiple ranges, similar to an **else if** ladder in procedural languages.

```
?- grade(85, G).
G = 'A'.

?- grade(50, G).
G = 'C'.

?- grade(10, G).
G = 'F'.
```

20.7 Common mistakes

Common Mistakes

- Forgetting the outer parentheses around the whole if-then-else expression, which can cause it to be parsed incorrectly due to operator precedence.
- Writing **Condition** **-> ThenGoal** *without* an else branch and being surprised when it simply fails (rather than doing nothing) if **Condition** does not hold.
- Overusing if-then-else where separate clauses would be clearer and more idiomatic — Prolog style often prefers multiple small clauses (as in most of this book’s earlier chapters) over deeply chained if-then-else ladders, except for cases like the grading example where the ladder structure is genuinely the clearest option.
- Expecting backtracking into the condition of an if-then-else to find alternative solutions — it will not, since the condition is committed to once it first succeeds.

Chapter Summary

- `( Condition -> Then ; Else )` provides explicit if-then-else logic in Prolog.
- The condition is committed to on its first success — no backtracking into it occurs.
- Chained if-then-else expressions can express multi-branch logic similar to `else if` ladders.
- Idiomatic Prolog often prefers multiple clauses over if-then-else, but the construct is valuable for simple, self-contained conditional logic.

Exercises

1. Rewrite the `classify/2` predicate from Section 20.4 using two separate clauses (with the appropriate comparisons) instead of if-then-else. Confirm both versions behave the same.
2. Write a predicate `bmi_category(BMI, Category)` using chained if-then-else, following standard BMI ranges.
3. Explain what happens (and why) when you query `?- (1 > 2 -> writeln(yes) ; true), writeln(done).`
4. Why does if-then-else commit to the first solution of its condition, and how does this differ from ordinary rule matching?
5. Extend the grading example to include a grade for scores between 40 and 60 called `'D'` instead of merging it into `'C'`.

Chapter 21

Finding and Searching Solutions

21.1 Search problems in Prolog

Many interesting problems can be phrased as: “find a value (or combination of values) that satisfies certain conditions.” Prolog’s built-in backtracking makes it naturally suited to this style of problem solving, often called **search**.

21.2 Generating candidates

The first step in solving a search problem in Prolog is usually to write a goal that can produce (**generate**) candidate values, typically using `member/2`, `between/3`, or facts.

```
?- between(1, 3, X).  
X = 1 ;  
X = 2 ;  
X = 3.
```

21.3 Testing candidates

The second step is to **test** each candidate against the conditions of the problem, using ordinary goals — arithmetic comparisons, further predicate calls, or negation.

21.4 Generate-and-test

Concept

The **generate-and-test** pattern combines these two steps: generate a candidate, then test it; if the test fails, Prolog automatically backtracks into the generator to try the next candidate.

Example: Generate-and-Test: Sum to Target

```
sums_to(X, Y, Target) :-  
    between(1, 10, X),  
    between(1, 10, Y),  
    X + Y =:= Target.
```

```
?- sums_to(X, Y, 5).  
X = 1, Y = 4 ;  
X = 2, Y = 3 ;  
X = 3, Y = 2 ;  
X = 4, Y = 1.
```

21.5 Constraint-style thinking

Even without using the specialized CLP(FD) library (Chapter 26), you can think about problems in terms of constraints: “X and Y must be between 1 and 10, AND $X + Y$ must equal 5.” Writing the constraints as separate goals, in a sensible order (cheaper/more restrictive checks first, when possible), keeps programs both readable and efficient.

21.6 Simple puzzle solving

Example: Two Numbers Puzzle

Problem: Find two numbers between 1 and 20 whose product is 48 and whose sum is 14.

```
puzzle(X, Y) :-
    between(1, 20, X),
    between(1, 20, Y),
    X * Y == 48,
    X + Y == 14.
```

```
?- puzzle(X, Y).
X = 6, Y = 8 ;
X = 8, Y = 6.
```

21.7 Number problems

Example: Pythagorean Triples

```
pythagorean(A, B, C) :-
    between(1, 20, A),
    between(1, 20, B),
    between(1, 20, C),
    A <= B,
    A*A + B*B == C*C.
```

```
?- pythagorean(A, B, C).
A = 3, B = 4, C = 5 ;
A = 6, B = 8, C = 10 ;
A = 5, B = 12, C = 13 ;
...
```

(Note the $A \leq B$ condition avoids reporting the same triple twice in swapped order.)

21.8 Logic puzzles

Example: A Tiny Logic Puzzle

Three friends, Alice, Bob, and Carl, each like a different fruit: apple, banana, or cherry. Alice does not like banana. Bob does not like apple. Who likes what?

```
likes(Alice, Bob, Carl) :-
    member(Alice, [apple, banana, cherry]),
```

```

member(Bob, [apple, banana, cherry]),
member(Carl, [apple, banana, cherry]),
Alice \= banana,
Bob \= apple,
Alice \= Bob,
Bob \= Carl,
Alice \= Carl.

```

```

?- likes(A, B, C).
A = apple, B = cherry, C = banana ;
A = cherry, B = banana, C = apple.

```

21.9 Practical search examples

Generate-and-test scales up naturally to classic AI puzzles like Sudoku and N-Queens (Chapter 28), though for larger problems the specialized CLP(FD) library (Chapter 26) becomes much more efficient, since it prunes impossible branches early instead of generating every combination blindly.

Common Mistakes

- Generating an enormous number of candidates before testing them, causing very slow performance — order goals so that cheaper, more restrictive tests run as early as possible.
- Forgetting inequality constraints (like `Alice \= Bob`) needed to avoid nonsensical or duplicate solutions in assignment-style puzzles.
- Using unbounded generators (e.g. forgetting the upper limit in `between/3`) leading to potentially infinite search.

Chapter Summary

- Generate-and-test combines a candidate generator with a set of test conditions, relying on Prolog's automatic backtracking.
- `between/3` and `member/2` are the most common generators.
- Ordering goals wisely (restrictive tests early) improves performance significantly.
- This pattern scales from small puzzles up to full constraint-satisfaction problems.

Exercises

1. Modify the two-numbers puzzle (Section 21.6) to find numbers whose product is 36 and whose sum is 13.
2. Explain why placing `A =< B` before or after the arithmetic test in the Pythagorean triple example could affect performance.
3. Extend the fruit-liking puzzle to include a fourth friend and fruit, adding at least one new constraint.
4. Write a generate-and-test predicate that finds all three-digit numbers whose digits sum to 10 (hint: use `between/3` three times for each digit).
5. Why is generate-and-test sometimes inefficient for larger problems, and what technique (pre-viewed in this chapter) addresses that inefficiency?

Chapter 22

Practical Prolog Programs

This chapter builds several complete, working programs from scratch, each following the same structure: problem statement, requirements, knowledge representation, complete code, line-by-line explanation, sample queries, expected output, how the reasoning works, and possible improvements.

22.1 Family relationship system

Problem statement: Build a system that can answer questions about family relationships (parent, sibling, grandparent) given basic parent/gender facts.

Requirements: Store parent and gender facts; derive sibling, grandparent, and ancestor relationships.

Knowledge representation: `parent(Parent, Child)`, `male(X)`, `female(X)`.

```
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).

male(tom).
male(bob).
female(liz).
female(ann).
female(pat).

father(X, Y) :-parent(X, Y), male(X).
mother(X, Y) :-parent(X, Y), female(X).

sibling(X, Y) :-
    parent(P, X),
    parent(P, Y),
    X \= Y.

grandparent(X, Z) :-
    parent(X, Y),
    parent(Y, Z).

ancestor(X, Y) :-parent(X, Y).
ancestor(X, Y) :-
    parent(X, Z),
    ancestor(Z, Y).
```

Line-by-line explanation: `father/2` and `mother/2` narrow `parent/2` by gender. `sibling/2` finds a shared parent while excluding matching the same person to itself. `grandparent/2` chains two `parent` steps. `ancestor/2` is defined recursively: either directly a parent, or a parent of some ancestor further down the chain.

Sample queries and expected output:

```
?- sibling(ann, pat).
true.
```

```
?- grandparent(tom, ann).
true.

?- ancestor(tom, ann).
true.
```

How the reasoning works: `ancestor/2` demonstrates recursive rule definition (Chapter 12): the base case handles direct parenthood, and the recursive case follows the parent chain arbitrarily deep.

Possible improvements: add `cousin/2`, handle multiple marriages, or add a **sex-neutral parent** relation that also derives from separate `mother/father` facts.

22.2 Student information system

Problem: Store and query student records including department and GPA.

```
student(rahim, cse, 3.8).
student(karim, eee, 3.2).
student(nadia, cse, 3.9).

honor_roll(Name) :-
    student(Name, _, GPA),
    GPA >= 3.5.

students_in(Dept, Name) :-
    student(Name, Dept, _).
```

```
?- honor_roll(X).
X = rahim ;
X = nadia.

?- students_in(cse, X).
X = rahim ;
X = nadia.
```

Possible improvements: add course enrollment facts and a predicate to compute a student's overall standing across multiple semesters.

22.3 Employee database

```
employee(nadia, engineering, 75000).
employee(hasan, sales, 55000).
employee(farah, engineering, 68000).

department_employees(Dept, Name) :-
    employee(Name, Dept, _).

average_salary(Dept, Avg) :-
    findall(S, employee(_, Dept, S), Salaries),
    sum_list(Salaries, Total),
    length(Salaries, N),
    N > 0,
    Avg is Total / N.
```

```
?- average_salary(engineering, Avg).
Avg = 71500.0.
```

(`findall/3` is covered fully in Chapter 25; here it collects every salary in a department into a list for aggregation.)

22.4 Simple recommendation system

```
likes(alice, action).
likes(alice, comedy).
likes(bob, drama).

recommend(Person, Genre) :-
    likes(Person, Genre).
```

```
?- recommend(alice, G).
G = action ;
G = comedy.
```

22.5 Movie recommendation system

```
movie(inception, scifi).
movie(the_matrix, scifi).
movie(the_hangover, comedy).

genre_preference(alice, scifi).
genre_preference(bob, comedy).

recommend_movie(Person, Movie) :-
    genre_preference(Person, Genre),
    movie(Movie, Genre).
```

```
?- recommend_movie(alice, M).
M = inception ;
M = the_matrix.
```

Possible improvements: rank recommendations by rating, or combine multiple genre preferences per person.

22.6 Course prerequisite checker

```
prerequisite(calculus1, calculus2).
prerequisite(calculus2, calculus3).
prerequisite(intro_cs, data_structures).

can_take(Student, Course) :-
    prerequisite(Pre, Course),
    completed(Student, Pre).
can_take(_, Course) :-
    \+ prerequisite(_, Course).

completed(rahim, calculus1).
completed(rahim, intro_cs).
```

```
?- can_take(rahim, calculus2).
true.

?- can_take(rahim, calculus3).
false.
```

Possible improvements: support multiple required prerequisites per course using `forall/2` (Chapter 25) instead of a single prerequisite fact.

22.7 Medical-style symptom reasoning example

```
symptom(patient1, fever).
symptom(patient1, cough).
symptom(patient2, headache).

diagnosis(Patient, flu) :-
    symptom(Patient, fever),
    symptom(Patient, cough).

diagnosis(Patient, migraine) :-
    symptom(Patient, headache).
```

```
?- diagnosis(patient1, D).
D = flu.
```

Note: this is a simplified educational example only, not a real medical diagnostic tool.

22.8 Restaurant recommendation system

```
restaurant(spice_hut, indian, cheap).
restaurant(sushi_place, japanese, expensive).
restaurant(pasta_house, italian, moderate).

preference(alice, indian).
preference(bob, italian).

recommend_restaurant(Person, Restaurant) :-
    preference(Person, Cuisine),
    restaurant(Restaurant, Cuisine, _).
```

```
?- recommend_restaurant(alice, R).
R = spice_hut.
```

22.9 Simple expert system

```
animal(X, bird) :-
    has_feathers(X).
animal(X, mammal) :-
    has_fur(X).

has_feathers(tweety).
has_fur(rex).
```

```
?- animal(tweety, T).
T = bird.
```

Possible improvements: generalize into a full rule-based expert system with certainty factors and explanation traces, as discussed further in Chapter 23.

22.10 Simple chatbot-style rule system

```
response(hello, 'Hello! How can I help you?').
response(bye, 'Goodbye! Have a great day.').
response(_, "I don't understand that.").
```

```
chat(Input) :-
    response(Input, Reply),
    writeln(Reply), !.
```

```
?- chat(hello).
Hello! How can I help you?
true.

?- chat(unknown_input).
I don't understand that.
true.
```

(The cut ! prevents backtracking into the catch-all clause when an earlier match already succeeded; fully explained in Chapter 25.)

Chapter Summary

- Every practical Prolog program follows the same core pattern: represent knowledge as facts, derive new knowledge with rules, and query for answers.
- Aggregation (like averaging salaries) typically uses `findall/3` together with list predicates like `sum_list/2` and `length/2`.
- Even “chatbot” and “expert system” behavior in Prolog reduces to matching facts and rules against an input.

Exercises

1. Extend the family system with a `cousin/2` predicate.
2. Add a rating field to the movie database and modify `recommend_movie/2` to only recommend movies rated above a threshold.
3. Extend the course prerequisite checker to require *all* prerequisites of a course, not just one.
4. Add two more responses to the chatbot system, including one that uses a partial keyword check instead of exact matching.
5. Combine the restaurant and student systems into one program with a shared `person/1` fact base.

Chapter 23

Prolog for Artificial Intelligence

23.1 Why Prolog is relevant to AI

Prolog was designed from the outset for symbolic reasoning, which made it a natural fit for early AI research. Even in the era of modern machine learning, Prolog remains relevant wherever a problem is naturally expressed in terms of rules, facts, and logical relationships rather than statistical patterns learned from large datasets.

23.2 Knowledge representation

Facts and rules are, quite literally, a way of representing knowledge — this is why Prolog is often used to teach and prototype **knowledge representation**, the AI subfield concerned with how information about the world can be encoded so that a computer can reason about it.

23.3 Rule-based systems

A **rule-based system** makes decisions by matching input data against a set of “if-then” rules. Prolog rules map directly onto this idea, making the language a natural fit for implementing such systems (as seen in Section 22.9’s simple expert system).

23.4 Expert systems

An **expert system** attempts to capture the decision-making process of a human expert in a particular domain (medicine, finance, engineering) as a collection of rules. Prolog’s ability to chain rules together and explain *why* a conclusion was reached (by tracing which rules fired) made it historically popular for building expert systems.

23.5 Inference

Inference is the process of deriving new facts from existing ones using logical rules — exactly what happens every time Prolog satisfies a rule’s body to prove its head. Prolog’s resolution mechanism (Chapter 14) is a general-purpose inference engine.

23.6 Search

As explored in Chapter 21, many AI problems (puzzles, planning, pathfinding) can be reduced to searching through a space of possible states for one that satisfies certain goals. Prolog’s backtracking search engine provides this capability without any extra library code.

23.7 Natural language processing

Prolog has a long history in **natural language processing** (NLP), particularly through Definite Clause Grammars (Chapter 27), which let you describe the grammar of a language directly as Prolog rules and use Prolog’s search engine to parse sentences.

23.8 Symbolic AI

Concept

Symbolic AI is the branch of artificial intelligence that represents knowledge using explicit symbols (words, facts, rules) and manipulates them according to formal logical rules, in contrast to **sub-symbolic** approaches like neural networks, which represent knowledge implicitly as learned numerical weights.

Prolog is one of the flagship languages of symbolic AI, alongside Lisp.

23.9 Prolog vs modern machine learning

Aspect	Prolog (Symbolic AI)	Machine Learning
Knowledge source	Explicitly written by humans (facts, rules)	Learned automatically from data
Reasoning style	Logical deduction	Statistical pattern recognition
Explainability	High — you can trace exactly which rules fired	Often low (“black box” models)
Handling uncertainty	Limited by default (true/false)	Naturally probabilistic
Best suited for	Well-defined rules, structured domains	Pattern recognition, large unstructured data (images, text, speech)

23.10 When Prolog is a good choice

Prolog (or logic programming more generally) tends to shine when:

- The problem’s rules are well understood and can be stated explicitly (e.g. regulations, prerequisite chains, grammar rules).
- You need to explain *why* a conclusion was reached, not just what the conclusion is.
- The problem is naturally a search or constraint-satisfaction problem (scheduling, puzzles, configuration).
- You are prototyping or teaching logical reasoning concepts.

It is generally a poor choice for tasks involving image recognition, speech processing, or any domain where the “rules” are too complex or numerous to write down by hand — such problems are today handled far more effectively by machine learning.

Chapter Summary

- Prolog belongs to the symbolic AI tradition: reasoning by manipulating explicit facts and rules.
- It naturally supports knowledge representation, inference, rule-based/expert systems, and search.
- Prolog and machine learning are complementary tools, each suited to different kinds of problems.

Exercises

1. Give an example of a real-world problem better suited to Prolog than to machine learning, and explain why.
2. Give an example of a real-world problem better suited to machine learning than to Prolog, and explain why.
3. Explain, in your own words, what “explainability” means, and why Prolog systems tend to have it while many machine-learning models do not.
4. Research (or recall) one historical expert system and identify which parts of it map onto Prolog facts versus rules.

5. Why might a modern AI system combine both symbolic (Prolog-like) and statistical (machine-learning) components?

Chapter 24

Debugging Prolog

24.1 Common syntax errors

Syntax errors are caught immediately when a file is loaded, and SWI-Prolog reports the approximate line number. The most frequent causes are: a missing period, mismatched parentheses or brackets, and an unquoted atom that was meant to start with an uppercase letter.

24.2 Undefined predicate

```
?- studnet(rahim).  
ERROR: Unknown procedure: studnet/1
```

This error means Prolog has no facts or rules at all for a predicate with that exact name and arity — very often a simple typo, as in this example (`studnet` instead of `student`).

24.3 Instantiation errors

```
?- X is Y + 1.  
ERROR: Arguments are not sufficiently instantiated
```

This occurs when a predicate (very commonly, arithmetic evaluation with `is`) requires a value to already be bound, but it is still an unbound variable.

24.4 Type errors

```
?- X is foo + 1.  
ERROR: Type error: 'evaluable' expected, found 'foo'
```

This occurs when a term of the wrong type is supplied where a specific type (like a number) is required.

24.5 Arithmetic errors

```
?- X is 5 / 0.  
ERROR: Arithmetic: evaluation error: 'zero_divisor'
```

Division by zero and similar undefined operations raise arithmetic evaluation errors.

24.6 Infinite recursion

If a recursive predicate lacks a reachable base case, or the recursive case never actually makes progress toward the base case, Prolog will recurse forever, eventually exhausting the stack:

```
count_up_bad(N) :-
    write(N), nl,
    N1 is N + 1,
    count_up_bad(N1). % no stopping condition!
```

Running `?- count_up_bad(1).` will print numbers forever (or until the process is interrupted or runs out of memory).

24.7 Unexpected false

If a query unexpectedly returns **false**, common causes include: a typo in a predicate or atom name, a missing fact, an argument order mismatch, or a rule condition that is stricter than intended.

24.8 Unexpected multiple answers

If a query returns more solutions than expected, common causes include: overlapping clauses that both match the same case, or missing constraints (like `X \= Y`) that should have ruled out duplicate or symmetric solutions (as seen in Chapter 13’s pair-generation example).

24.9 Debugging with trace

As introduced in Chapter 14, `trace/0` shows every **Call**, **Exit**, **Fail**, and **Redo** event as Prolog processes a query, letting you observe exactly where behavior diverges from what you expected.

```
?- trace.
?- grandparent(tom, X).
Call: (8) grandparent(tom, _1234) ?
Call: (9) parent(tom, _5678) ?
Exit: (9) parent(tom, bob) ?
...
```

Type `notrace.` to stop tracing.

24.10 Reading Prolog error messages

Prolog error messages generally follow the pattern: **ERROR: <context> <error type>: <description>**. Learning to read the error *type* (instantiation, type, existence, evaluation) quickly narrows down the likely cause, even before reading the full description.

24.11 Debugging checklist

When something goes wrong, work through this checklist:

1. Re-read the exact predicate and atom names for typos.
2. Check argument order matches your intended convention consistently.
3. Confirm every variable that should be bound before use (especially before **is** or arithmetic comparisons) actually is.
4. Check for a missing base case in any recursive predicate.
5. Use `trace/0` to watch the actual execution step by step.
6. Test each predicate in isolation with simple queries before combining it with others.
7. Check for missing inequality constraints if you’re getting duplicate or symmetric answers.

Common Mistakes

- Assuming an “Unknown procedure” error always means a bug in logic, rather than checking for a simple typo first.
- Ignoring the specific error *type* (instantiation vs type vs evaluation) and treating all errors the same way.
- Not testing small predicates individually before composing them into larger rules, making it hard to isolate where a bug originates.

Chapter Summary

- Prolog reports specific error types: instantiation errors, type errors, arithmetic errors, and undefined-procedure errors.
- Infinite recursion, unexpected **false**, and unexpected multiple answers are the most common “silent” bugs (no error message, just wrong results).
- **trace/0** is the single most powerful tool for understanding what Prolog is actually doing during a failing or misbehaving query.

Exercises

1. Deliberately introduce a typo into a predicate name in one of your earlier programs, run it, and read the resulting error message carefully.
2. Write a recursive predicate with a missing base case, run it with a small input, and observe what happens (be ready to interrupt it).
3. Use **trace/0** on the **sibling/2** predicate from Chapter 4 and describe the sequence of Call/Exit events.
4. Given a query that returns **false** unexpectedly, list three possible causes you would check first, based on the checklist in Section 24.11.
5. Explain the difference between an instantiation error and a type error, with an example of each.

Chapter 25

Advanced Prolog Concepts

25.1 Dynamic predicates

By default, Prolog treats the clauses loaded from a file as fixed for the duration of the program. A **dynamic predicate** is one you explicitly allow to be modified — facts added or removed — while the program runs. Declare this with:

```
:- dynamic(score/2).
```

25.2 assert/1

`assert/1` (and its more explicit variants `assertz/1`, `asserta/1`) adds a new fact or rule to the database at runtime.

```
?- assertz(score(alice, 90)).  
true.  
  
?- score(alice, X).  
X = 90.
```

`assertz/1` adds at the end of existing clauses for that predicate; `asserta/1` adds at the beginning.

25.3 retract/1

`retract/1` removes a specific matching fact or rule from the database.

```
?- retract(score(alice, 90)).  
true.  
  
?- score(alice, X).  
false.
```

25.4 retractall/1

`retractall/1` removes *all* facts matching a given pattern, without failing even if none exist.

```
?- retractall(score(alice, _)).  
true.
```

25.5 cut (!)

Concept

The **cut**, written **!**, is a special goal that, when executed, commits Prolog to all the choices made so far in the current clause — it discards any remaining choice points for the current predicate call, preventing backtracking from undoing them.

```
max(X, Y, X) :- X >= Y, !.  
max(_, Y, Y).
```

Once the first clause succeeds ($X \geq Y$), the cut prevents Prolog from ever trying the second clause for that same call, even under backtracking.

25.6 Why cut exists

The cut exists for two main reasons: to improve efficiency (by stopping Prolog from exploring branches you already know are unnecessary), and to enforce “if-then-else”-like determinism in situations where you want exactly one answer rather than several.

25.7 Green cut vs red cut

Concept

A **green cut** only improves efficiency and does not change the set of solutions a predicate can produce — removing it would still give correct (if slower) results. A **red cut** actually changes the logical meaning of the program — removing it would produce different (often incorrect or extra) answers. Red cuts should be used carefully and documented clearly, since they make code harder to reason about.

The `max/3` example above is a red cut: without it, calling `max(5, 5, X)` would backtrack into the second clause and incorrectly also offer $X = 5$ again via a different path — actually harmless here since the value is the same, but in general such cuts change behavior meaningfully.

25.8 fail-driven loops

Combining `fail/0` with backtracking can be used to iterate through every solution for its side effects (like printing), though `forall/2` (Section 25.9’s neighbor, actually built on `\+` and `findall`) is usually clearer:

```
print_all_students :-  
    student(Name, _, _),  
    writeln(Name),  
    fail.  
print_all_students.
```

This works because `fail` forces backtracking into `student(Name, _, _)` repeatedly until no more solutions remain, at which point the whole first clause fails and Prolog falls through to the second, always-true clause, ensuring the overall call still succeeds.

25.9 findall/3

`findall/3` collects *all* solutions of a goal into a list, without requiring manual backtracking.

```
?- findall(Name, student(Name, cse, _), Names).  
Names = [rahim, nadia].
```

25.10 bagof/3

`bagof/3` is similar to `findall/3`, but fails (rather than returning an empty list) if there are no solutions, and can group results by free variables using the `^` operator.

```
?- bagof(Name, Dept^student(Name, Dept, _), Names).  
Names = [rahim, karim, nadia].
```

25.11 setof/3

`setof/3` behaves like `bagof/3` but additionally sorts the results and removes duplicates.

```
?- setof(Dept, Name^student(Name, Dept, _), Depts).  
Depts = [cse, eee].
```

25.12 Higher-order programming

Concept

Higher-order programming refers to treating goals or predicates themselves as data that can be passed around and invoked dynamically, rather than being fixed at the time you write the code.

25.13 Meta-predicates

A **meta-predicate** is a predicate that takes another goal as an argument and calls it, such as `findall/3`, `forall/2`, or `call/1` (below).

25.14 call/1

`call/1` (and its variants `call/2`, `call/3`, etc.) invokes a goal that is passed as a term, optionally adding extra arguments.

```
?- call(writeln, hello).  
hello  
true.  
  
?- G = (X is 2+2), call(G).  
X = 4.
```

25.15 Definite Clause Grammars

Definite Clause Grammars (DCGs) provide a convenient notation (using `->` instead of `:-`) for describing grammars and parsing sequences, especially useful for natural language processing (Chapter 27).

25.16 Difference lists

A **difference list** represents a list as the difference between two lists, e.g. representing `[a,b,c]` as `L1 - L2` where `L1 = [a,b,c|L2]`. This technique allows appending to the “end” of a list in constant time, which is especially useful for efficient DCG-based parsing.

25.17 Accumulators

Concept

An **accumulator** is an extra argument threaded through a recursive predicate that carries a partial result forward, allowing the final result to be built incrementally rather than combined only after the recursion fully unwinds.

Example: Sum With an Accumulator

```
sum_list_acc(List, Sum) :-
    sum_list_acc(List, 0, Sum).

sum_list_acc([], Acc, Acc).
sum_list_acc([H|T], Acc, Sum) :-
    NewAcc is Acc + H,
    sum_list_acc(T, NewAcc, Sum).
```

```
?- sum_list_acc([1,2,3,4], S).
S = 10.
```

25.18 Tail recursion

Concept

A predicate is **tail recursive** if its recursive call is the very last goal executed in that clause, with no further computation needed afterward (as in `sum_list_acc/3` above, where the accumulator carries forward the running total instead of computing `Sum is H + RestSum` after the recursive call returns, as in Chapter 12’s non-accumulator version).

Tail-recursive predicates can be optimized by Prolog implementations to run in constant stack space, making them much more efficient for processing very long lists.

25.19 Performance considerations

- Prefer tail recursion with accumulators for predicates that process very large lists.
- Use cuts judiciously (understand green vs red cut) to eliminate unnecessary choice points.
- Order goals so cheaper or more restrictive checks run first (Chapter 21).
- Use `findall/3`/`setof/3` rather than manual fail-driven loops when collecting results, for clarity and safety.

Common Mistakes

- Overusing cuts without understanding whether they are green or red, leading to subtly broken logic.

- Forgetting to declare a predicate `dynamic` before using `assert/1` or `retract/1` on it, which raises a permission error in strict settings.
- Writing recursive predicates without accumulators for very large inputs, causing excessive stack usage or so-called “non-tail-recursive” slowdowns.

Chapter Summary

- `assert/1`, `retract/1`, and `retractall/1` let a program modify its own database at runtime.
- The cut (!) commits to choices already made, improving efficiency (green cut) or changing behavior deliberately (red cut).
- `findall/3`, `bagof/3`, and `setof/3` collect all solutions of a goal into a list, with different handling of empty results and duplicates.
- Accumulators enable efficient, tail-recursive predicates for processing large data.

Exercises

1. Explain the difference between `findall/3` and `setof/3` using an example with duplicate results.
2. Rewrite the `list_length/2` predicate from Chapter 12 using an accumulator, and explain why it is tail recursive.
3. Give an example of a green cut and a red cut, and explain the difference in your own words.
4. Use `assertz/1` to add three new facts to a dynamic predicate, then use `retractall/1` to clear them.
5. Write a small program using `call/2` to apply a user-supplied comparison predicate to two numbers.

Chapter 26

Constraint Logic Programming

26.1 What is constraint logic programming?

Concept

Constraint Logic Programming (CLP) extends ordinary logic programming by allowing variables to be restricted to a set of possible values (their **domain**) and letting Prolog automatically **prune** impossible combinations early, rather than generating every candidate blindly as in the generate-and-test pattern of Chapter 21.

The most commonly used variant is **CLP(FD)** — Constraint Logic Programming over Finite Domains — used for problems over integers with a bounded range.

26.2 CLP(FD)

In SWI-Prolog, load the library with:

```
:- use_module(library(clpfd)).
```

This gives access to constraint operators such as `#=`, `ins`, and `label/1`.

26.3 Constraints vs normal arithmetic

Ordinary arithmetic with `is` (Chapter 10) requires all variables on the right-hand side to already be bound — it cannot work “backwards”. CLP(FD) constraints, by contrast, can be stated *before* variables are bound, and Prolog will maintain them as more information becomes available, only fully resolving values at the end via **labeling**.

26.4 Domains

A variable’s **domain** is the set of values it is allowed to take. You assign or restrict domains using `ins/2`.

26.5 `ins/2`

```
?- X ins 1..10.  
X in 1..10.
```

This restricts `X` to integers from 1 to 10 (inclusive), without yet picking a specific value.

26.6 `#=`

The constraint equality operator. Unlike `is`, both sides may contain unbound (but domain-restricted) variables.

```
?- X #= Y + 1, X in 1..10.
X in 2..10,
Y in 1..9.
```

26.7 #\=

Constraint inequality — the two sides must never be equal.

```
?- X #\= Y, X in 1..2, Y in 1..2.
```

26.8 #<

Constraint “less than”.

26.9 #>

Constraint “greater than”.

26.10 all_distinct/1

Ensures every variable in a list takes a different value from all the others — extremely useful for puzzles like Sudoku and N-Queens.

```
?- L = [X, Y, Z], L ins 1..3, all_distinct(L), label(L).
L = [1, 2, 3] ;
L = [1, 3, 2] ;
...
```

26.11 labeling/2

Constraints alone only narrow down possibilities; `label/1` (or the more configurable `labeling/2`) is what actually forces Prolog to search for and commit to concrete values consistent with all constraints.

```
?- X in 1..5, Y in 1..5, X #< Y, label([X,Y]).
X = 1, Y = 2 ;
X = 1, Y = 3 ;
...
```

26.12 Example: Sudoku

Example: Sudoku Skeleton with CLP(FD)

```
:- use_module(library(clpfd)).

sudoku(Rows) :-
    length(Rows, 9),
    maplist(same_length(Rows), Rows),
    append(Rows, Vs),
    Vs ins 1..9,
```

```

maplist(all_distinct, Rows),
transpose(Rows, Columns),
maplist(all_distinct, Columns),
Rows = [A,B,C,D,E,F,G,H,I],
blocks(A, B, C),
blocks(D, E, F),
blocks(G, H, I),
maplist(label, Rows).

blocks([], [], []).
blocks([A,B,C|Bs1], [D,E,F|Bs2], [G,H,I|Bs3]) :-
    all_distinct([A,B,C,D,E,F,G,H,I]),
    blocks(Bs1, Bs2, Bs3).

```

This full solver demonstrates how CLP(FD) expresses the entire Sudoku ruleset (rows, columns, 3x3 blocks all distinct) as declarative constraints, letting the constraint solver find a valid assignment far more efficiently than plain generate-and-test.

26.13 Example: scheduling

Example: Simple Meeting Scheduling

```

:- use_module(library(clpfd)).

schedule(Start1, Start2) :-
    Start1 in 9..17,
    Start2 in 9..17,
    Start1 #\= Start2,
    label([Start1, Start2]).

```

This ensures two meetings, each starting somewhere between 9 and 17 (hours), never start at the same hour.

26.14 Example: cryptarithm

Example: SEND + MORE = MONEY

```

:- use_module(library(clpfd)).

send_more_money(S,E,N,D,M,O,R,Y) :-
    Vars = [S,E,N,D,M,O,R,Y],
    Vars ins 0..9,
    all_distinct(Vars),
    S #\= 0, M #\= 0,
    1000*S + 100*E + 10*N + D
    + 1000*M + 100*O + 10*R + E
    #= 10000*M + 1000*O + 100*N + 10*E + Y,
    label(Vars).

```

This is the classic “SEND + MORE = MONEY” cryptarithmic puzzle, where each letter represents a unique digit.

26.15 Practical applications

CLP(FD) and constraint programming more broadly are used in real-world applications such as employee shift scheduling, exam timetabling, resource allocation, and configuration problems (like choosing compatible components for a product), wherever the problem can be expressed as a set of variables with domains and constraints among them.

Common Mistakes

- Forgetting to load `library(clpfd)` before using constraint operators.
- Forgetting to call `label/1` (or `labeling/2`) at the end — without it, Prolog only reports the narrowed domains, not concrete solutions.
- Mixing plain arithmetic (`is`, `>`) with constraint operators (`#=`, `#>`) inconsistently, which can lose the benefits of constraint propagation.

Chapter Summary

- CLP(FD) lets you declare constraints on variables' possible integer values, pruning impossible branches automatically.
- `ins/2` declares a domain; `#=`, `#\=`, `#<`, `#>` declare relationships between (possibly still unbound) variables.
- `all_distinct/1` is essential for puzzles requiring unique assignments.
- `label/1` triggers the actual search for concrete solutions once constraints are declared.

Exercises

1. Modify the scheduling example to schedule three meetings, all at different hours between 9 and 17.
2. Explain why CLP(FD) is generally more efficient than plain generate-and-test for problems like Sudoku.
3. What would happen if you forgot to call `label/1` in the cryptarithm example? Predict the output.
4. Write a CLP(FD) program to find three consecutive integers that sum to 72.
5. Research another classic cryptarithmic puzzle and adapt the `SEND+MORE=MONEY` template to solve it.

Chapter 27

Natural Language Processing with Prolog

27.1 Introduction to NLP in Prolog

Prolog’s pattern-matching and backtracking search make it a historically important tool for **natural language processing** (NLP) — particularly for parsing sentences according to a formal grammar. This chapter introduces **Definite Clause Grammars** (DCGs), Prolog’s specialized notation for describing grammars.

27.2 Grammar representation

A grammar describes how larger language units (sentences) are built from smaller ones (phrases, words), typically using rules like:

```
sentence → noun_phrase, verb_phrase
```

DCGs let you write exactly this kind of rule directly in Prolog syntax.

27.3 Definite Clause Grammars

Concept

A **DCG rule** uses `->` instead of `:-` and describes how a grammar symbol can be broken down into a sequence of other symbols or words. Internally, Prolog translates DCG rules into ordinary clauses that thread a “list of remaining words” through each call — but you rarely need to think about that translation directly.

```
sentence ->noun_phrase, verb_phrase.
```

27.4 Sentence parsing

To parse a sentence, you use the built-in predicate `phrase/2`, which runs a DCG rule against a list of words:

```
?- phrase(sentence, [the, cat, sees, a, dog]).  
true.
```

27.5 Noun phrases

```
noun_phrase ->determiner, noun.  
determiner ->[the].  
determiner ->[a].  
noun ->[cat].
```

```
noun ->[dog].
```

27.6 Verb phrases

```
verb_phrase ->verb, noun_phrase.
verb ->[sees].
verb ->[chases].
```

27.7 Simple grammar

Example: A Complete Mini Grammar

```
sentence ->noun_phrase, verb_phrase.

noun_phrase ->determiner, noun.
determiner ->[the].
determiner ->[a].
noun ->[cat].
noun ->[dog].

verb_phrase ->verb, noun_phrase.
verb ->[sees].
verb ->[chases].
```

27.8 Example parser

```
?- phrase(sentence, [the, cat, sees, a, dog]).
true.

?- phrase(sentence, [a, dog, chases, the, cat]).
true.

?- phrase(sentence, [cat, the, sees, dog]).
false.
```

The third query fails because `[cat, the, sees, dog]` does not match the grammar's required word order (determiner before noun).

27.9 Querying grammar

DCGs can also be run “backwards” to *generate* valid sentences by leaving the word list unbound:

```
?- phrase(sentence, Sentence).
Sentence = [the, cat, sees, the, cat] ;
Sentence = [the, cat, sees, the, dog] ;
...
```

This works precisely because DCG rules are just ordinary Prolog clauses under the hood, and Prolog's search engine can run any clause in either direction, exactly like `append/3` in Chapter 11.

27.10 Practical NLP example

Example: Adding Adjectives

```
noun_phrase -> determiner, adjective, noun.  
noun_phrase -> determiner, noun.  
adjective -> [big].  
adjective -> [small].
```

```
?- phrase(sentence, [the, big, dog, chases, a, small, cat]).  
true.
```

Common Mistakes

- Forgetting that DCG rules use `->`, not `:-` — mixing the two will not work as intended.
- Forgetting to call DCG rules through `phrase/2` (or `phrase/3`) rather than calling them directly like ordinary predicates.
- Writing overlapping grammar rules without considering how they interact under backtracking, which can lead to unexpectedly many (or few) valid parses.

Chapter Summary

- DCGs (`->`) provide a natural notation for grammar rules in Prolog, translated automatically into ordinary clauses.
- `phrase/2` runs a DCG rule against a list of words to check or generate valid sequences.
- Because DCG rules are just Prolog clauses, they inherit backtracking search, letting the same grammar both parse and generate sentences.

Exercises

1. Extend the mini grammar to support plural nouns like `cats` and `dogs` together with a plural determiner `some`.
2. Add an adverb category (e.g. `quickly`) that can optionally modify the verb phrase.
3. Use `phrase/2` with an unbound word list to generate five different valid sentences from the grammar in Section 27.11.
4. Explain, in your own words, why a DCG rule with `->` translates naturally into ordinary Prolog clauses with hidden extra arguments.
5. Write a small grammar for simple yes/no questions, such as “does the cat see the dog”.

Chapter 28

Problem Solving with Prolog

This chapter is a structured problem set that progresses from beginner to advanced, giving you a chance to apply everything learned so far. Solutions are sketched briefly; you are encouraged to write and test your own versions before reading them.

Beginner Problems

Even/odd

```
is_even(N) :-N mod 2 == 0.  
is_odd(N) :-N mod 2 == 1.
```

Maximum of two numbers

```
max2(X, Y, X) :-X >= Y, !.  
max2(_, Y, Y).
```

Maximum of three numbers

```
max3(X, Y, Z, Max) :-  
    max2(X, Y, M1),  
    max2(M1, Z, Max).
```

Factorial

See Chapter 12, Section 12.6.

Fibonacci

See Chapter 12, Section 12.7.

Sum of list

See Chapter 12, Section 12.10.

Length of list

See Chapter 12, Section 12.11.

Member

See Chapter 11, Section 11.9.

Reverse

See Chapter 12, Section 12.13.

Intermediate Problems

Sorting

```
insert_sorted(X, [], [X]).
insert_sorted(X, [H|T], [X,H|T]) :-X <= H.
insert_sorted(X, [H|T], [H|R]) :-X > H, insert_sorted(X, T, R).

insertion_sort([], []).
insertion_sort([H|T], Sorted) :-
    insertion_sort(T, SortedT),
    insert_sorted(H, SortedT, Sorted).
```

Searching

```
find_index(X, List, Index) :-
    nth0(Index, List, X).
```

Duplicate removal

```
remove_duplicates([], []).
remove_duplicates([H|T], [H|R]) :-
    \+ member(H, T),
    remove_duplicates(T, R).
remove_duplicates([H|T], R) :-
    member(H, T),
    remove_duplicates(T, R).
```

Palindrome

```
is_palindrome(List) :-
    reverse(List, List).
```

Family relationships

See Chapter 22, Section 22.1.

Graph traversal

```
edge(a, b). edge(b, c). edge(c, d).

path(X, Y) :-edge(X, Y).
path(X, Y) :-edge(X, Z), path(Z, Y).
```

Path finding

```
find_path(X, X, [X]).
find_path(X, Y, [X|Path]) :-
    edge(X, Z),
    find_path(Z, Y, Path).
```

Permutations

Prolog provides `permutation/2` built in:

```
?- permutation([1,2,3], P).
P = [1,2,3] ;
P = [1,3,2] ;
...
```

Advanced Problems

Sudoku

See Chapter 26, Section 26.12.

N-Queens

Example: N-Queens with CLP(FD)

```
:- use_module(library(clpfd)).

queens(N, Queens) :-
    length(Queens, N),
    Queens ins 1..N,
    all_distinct(Queens),
    safe_queens(Queens),
    label(Queens).

safe_queens([]).
safe_queens([Q|Qs]) :-
    safe_queens(Qs, Q, 1),
    safe_queens(Qs).

safe_queens([], _, _).
safe_queens([Q|Qs], Q0, D) :-
    Q0 #\= Q,
    abs(Q0 - Q) #\= D,
    D1 is D + 1,
    safe_queens(Qs, Q0, D1).
```

```
?- queens(8, Qs).
Qs = [1, 5, 8, 6, 3, 7, 2, 4] ;
...
```

Cryptarithm

See Chapter 26, Section 26.14.

Scheduling

See Chapter 26, Section 26.13.

Constraint problems

See Chapter 26 in full for the underlying technique used to solve all problems in this section.

Expert system

See Chapter 22, Section 22.9, and Chapter 23 for design principles behind larger expert systems.

Chapter Summary

- This chapter's problems are meant to be practiced hands-on, not just read.
- Beginner problems reinforce recursion and list basics (Chapters 11-12); intermediate problems combine recursion with search (Chapters 13, 21); advanced problems rely on constraint logic programming (Chapter 26).
- Many built-in predicates (`permutation/2`, `nth0/3`) can shortcut problems you might otherwise solve manually — always check if SWI-Prolog already provides what you need.

Exercises

1. Implement `is_palindrome/1` without using the built-in `reverse/2`, writing your own reversal logic.
2. Extend the graph traversal example to detect and avoid infinite loops in a graph containing a cycle.
3. Modify insertion sort to sort in descending order instead of ascending.
4. Solve the 6-queens problem using the template in this chapter and report one valid solution.
5. Design your own intermediate-level problem (not listed here) and solve it using techniques from this book.

Chapter 29

Prolog Programming Patterns

Having worked through facts, rules, recursion, backtracking, and constraints, you have now met the individual mechanisms Prolog provides. This chapter names and summarizes the recurring **patterns** you have already been using throughout the book, so you can recognize and apply them deliberately in new problems.

29.1 Recursive pattern

Shape: a base case handling the simplest input, and a recursive case that reduces the problem toward the base case.

```
solve([], BaseResult).
solve([H|T], Result) :-
    solve(T, PartialResult),
    combine(H, PartialResult, Result).
```

Used in: factorial, list length, sum of list (Chapter 12).

29.2 Generate-and-test pattern

Shape: a generator goal proposes candidates; a test goal filters them; backtracking automatically retries the generator when a candidate fails the test.

```
solution(X) :-
    candidate(X),
    valid(X).
```

Used in: number puzzles, logic puzzles (Chapter 21).

29.3 Accumulator pattern

Shape: an extra argument threads a partial result forward through the recursion, so the final answer is ready as soon as the base case is reached, without further post-processing.

```
solve(List, Result) :- solve(List, InitialAcc, Result).
solve([], Acc, Acc).
solve([H|T], Acc, Result) :-
    update(Acc, H, NewAcc),
    solve(T, NewAcc, Result).
```

Used in: tail-recursive sum, efficient list length (Chapter 25).

29.4 Search pattern

Shape: explore a space of states, moving from one state to a neighboring state, until a goal state is reached.

```
path(X, X, [X]).
path(X, Y, [X|Rest]) :-
    move(X, Z),
    path(Z, Y, Rest).
```

Used in: graph traversal and path finding (Chapter 28).

29.5 Knowledge-base pattern

Shape: facts describe raw data; rules describe derived relationships built on top of that data.

```
fact(a, b).
derived(X, Y) :- fact(X, Z), fact(Z, Y).
```

Used in: family databases, student databases (Chapters 3-4, 22).

29.6 Rule-engine pattern

Shape: a collection of “if condition then conclusion” rules, matched against a set of asserted facts about a particular situation, often built with dynamic predicates (Chapter 25).

```
conclusion(X, Diagnosis) :-
    condition1(X),
    condition2(X),
    Diagnosis = some_result.
```

Used in: expert systems (Chapters 22-23).

29.7 List-processing pattern

Shape: decompose a list into head and tail, process the head, and recurse on the tail, building a new list or an aggregate result.

```
process([], []).
process([H|T], [H2|T2]) :-
    transform(H, H2),
    process(T, T2).
```

Used in: filtering, mapping, and transforming lists (Chapter 11).

29.8 Graph-processing pattern

Shape: represent nodes and edges as facts, then define reachability or path-finding rules recursively over the edge relation, taking care to avoid infinite loops in cyclic graphs (often by tracking visited nodes).

```
edge(a, b).
reachable(X, Y) :- edge(X, Y).
reachable(X, Y) :- edge(X, Z), reachable(Z, Y).
```

29.9 Constraint-solving pattern

Shape: declare variables with domains, state relationships between them as constraints, and finish with a labeling step to extract concrete solutions.

```
solve(Vars) :-  
    Vars ins 1..N,  
    all_distinct(Vars),  
    apply_constraints(Vars),  
    label(Vars).
```

Used in: Sudoku, N-Queens, cryptarithms, scheduling (Chapter 26).

Chapter Summary

- Nearly every Prolog program you write will be built from combinations of these nine patterns.
- Recognizing which pattern (or combination of patterns) fits a new problem is often the fastest path to a working solution.
- Patterns are not rigid templates — feel free to blend them, as many of the practical programs in Chapter 22 do.

Exercises

1. Identify which pattern(s) the family relationship system from Chapter 22 uses, and explain why.
2. Rewrite the generate-and-test pattern example as a constraint-solving pattern using CLP(FD), for a small domain of your choice.
3. Combine the list-processing pattern and the accumulator pattern to write a predicate that computes a running total alongside a transformed list.
4. Explain why the graph-processing pattern needs special care to avoid infinite loops, and describe one way to prevent them.
5. Choose one problem from Chapter 28 and identify every pattern from this chapter that it relies on.

Chapter 30

Final Project: University Academic Advisor Expert System

This final chapter brings together everything covered in the book — facts, rules, recursion, lists, negation, aggregation, and dynamic predicates — into one complete, realistic project: an expert system that advises students on which courses they are eligible to take and whether they are on track to graduate.

30.1 System design

The system will:

- Store student records, including completed courses and grades.
- Store course information, including prerequisites and credit values.
- Track each student's declared interests (for elective recommendations).
- Determine course eligibility based on completed prerequisites.
- Recommend elective courses matching the student's interests.
- Warn about low GPA or missing prerequisites.
- Check overall graduation requirements (minimum total credits).

30.2 Knowledge representation

```
% student(Name, GPA).
student(rahim, 3.6).
student(karim, 2.1).

% completed(Student, Course, Grade).
completed(rahim, intro_cs, 'A').
completed(rahim, data_structures, 'A').
completed(rahim, calculus1, 'B').
completed(karim, intro_cs, 'C').

% course(Course, Credits).
course(intro_cs, 3).
course(data_structures, 3).
course(algorithms, 3).
course(calculus1, 4).
course(calculus2, 4).
course(ai_intro, 3).
course(databases, 3).

% prerequisite(Course, Requires).
prerequisite(data_structures, intro_cs).
prerequisite(algorithms, data_structures).
prerequisite(calculus2, calculus1).
prerequisite(ai_intro, algorithms).
```

```
prerequisite(databases, data_structures).

% interest(Student, Topic).
interest(rahim, artificial_intelligence).
interest(karim, databases).

% elective(Course, Topic).
elective(ai_intro, artificial_intelligence).
elective(databases, databases).
```

30.3 Complete Prolog source code

```
% ----- Eligibility -----
has_completed(Student, Course) :-
    completed(Student, Course, _).

eligible(Student, Course) :-
    \+ has_completed(Student, Course),
    forall(prerequisite(Course, Req), has_completed(Student, Req)).

% forall(Condition, Action) succeeds if Action holds
% for every solution of Condition; introduced here as a
% practical companion to findall/3 from Chapter 25.

% ----- Recommendations -----
recommend_elective(Student, Course) :-
    interest(Student, Topic),
    elective(Course, Topic),
    eligible(Student, Course).

% ----- Warnings -----
low_gpa_warning(Student) :-
    student(Student, GPA),
    GPA < 2.5.

missing_prereq_warning(Student, Course) :-
    course(Course, _),
    \+ has_completed(Student, Course),
    prerequisite(Course, Req),
    \+ has_completed(Student, Req).

% ----- Graduation Requirement -----
total_credits(Student, Total) :-
    findall(Credits,
        (completed(Student, Course, _), course(Course, Credits)),
        CreditList),
    sum_list(CreditList, Total).

can_graduate(Student) :-
    total_credits(Student, Total),
    Total >= 120.

graduation_status(Student, Status) :-
    ( can_graduate(Student)
    -> Status = eligible_to_graduate
    ; Status = not_yet_eligible
```

```
).
```

30.4 Detailed explanation

- `has_completed/2` is a small helper predicate wrapping `completed/3`, hiding the `grade` argument when it is irrelevant — an example of the predicate-design principles from Chapter 15.
- `eligible/2` uses `forall/2` (built on negation and `findall/3`, Chapter 25) to require that *every* prerequisite of a course has been completed, unlike the simpler single-prerequisite checker in Chapter 22.
- `recommend_elective/2` combines a student's declared interest with the eligibility check, demonstrating the recommendation-system pattern from Chapter 22 layered on top of the rule-engine pattern from Chapter 29.
- `low_gpa_warning/1` and `missing_prereq_warning/2` are simple rule-based warnings, in the spirit of an expert system (Chapter 23).
- `total_credits/2` uses `findall/3` to aggregate credit values across every completed course, then `sum_list/2` to total them — the same aggregation pattern used for average salary in Chapter 22.
- `graduation_status/2` uses if-then-else (Chapter 20) to translate the graduation check into a readable status atom.

30.5 Sample queries

```
?- eligible(rahim, algorithms).
true.

?- eligible(karim, algorithms).
false.

?- recommend_elective(rahim, X).
X = ai_intro.

?- low_gpa_warning(karim).
true.

?- missing_prereq_warning(karim, algorithms).
true.

?- total_credits(rahim, T).
T = 10.

?- graduation_status(rahim, Status).
Status = not_yet_eligible.
```

30.6 Expected outputs

As shown above: Rahim, having completed `intro_cs` and `data_structures`, is eligible for `algorithms`, while Karim (who has only completed `intro_cs`) is not, since `algorithms` requires `data_structures` first. Rahim's interest in artificial intelligence correctly leads to a recommendation of `ai_intro` once he becomes eligible for it. Neither student has yet accumulated 120 credits, so both currently show `not_yet_eligible`.

30.7 Execution flow

When `recommend_elective(rahim, X)` is queried, Prolog: (1) finds Rahim's interest, `artificial_intelligence`; (2) finds the matching elective, `ai_intro`; (3) checks eligibility for `ai_intro`, which recursively checks that its prerequisite `algorithms` is completed, which in turn was only satisfiable because `data_structures` (its own prerequisite) had already been completed. This chains together exactly the kind of layered rule reasoning introduced across Chapters 4, 12, and 25.

30.8 Extension ideas

- Add support for multiple sections of the same course with different terms/semesters.
- Add a full `degree_requirement(Program, RequiredCourses)` fact set and check completion against a specific program rather than a flat 120-credit threshold.
- Track prerequisite chains transitively (using recursion, as in Chapter 22's `ancestor/2`) to warn students several semesters in advance about long prerequisite chains.
- Add a simple command-line menu (using the I/O techniques from Chapter 19) so an advisor can interactively query the system.
- Migrate the eligibility logic into CLP(FD) (Chapter 26) to automatically generate an optimal multi-semester course plan given credit-per-semester limits.

Chapter Summary

- This project demonstrates how the individual techniques from every previous chapter — facts, rules, recursion, aggregation, negation, if-then-else, and predicate design — combine into a single, realistic, rule-based application.
- Expert-system-style reasoning in Prolog is fundamentally the same mechanism used in the very first `student/1` and `smart/1` example in Chapter 1 — just scaled up with more facts, more rules, and more careful predicate design.

Exercises

1. Add a new student and a full set of completed courses, then test every predicate in this chapter against them.
2. Extend `missing_prereq_warning/2` to list *all* missing prerequisites for a course, not just report true/false for one.
3. Implement transitive prerequisite checking so a student is warned about prerequisites-of-prerequisites they have not yet completed.
4. Add a `degree_requirement/2` fact set for a specific major and rewrite `can_graduate/1` to check against it instead of a flat credit total.
5. Build a simple interactive menu (Chapter 19 style) that lets a user type a student's name and see all recommendations and warnings printed automatically.

Appendix A

Prolog Syntax Cheat Sheet

Syntax	Meaning	Example	Explanation
<code>pred(a,b).</code>	Fact	<code>parent(tom,bob).</code>	States an unconditional truth (Ch. 3).
<code>H :- B.</code>	Rule	<code>happy(X):-rich(X).</code>	H is true if B is true (Ch. 4).
<code>?- goal.</code>	Query	<code>?- happy(bill).</code>	Asks Prolog to prove a goal (Ch. 6).
<code>X, Name</code>	Variable	<code>parent(tom,X).</code>	Placeholder for an unknown value (Ch. 5).
<code>_</code>	Anonymous variable	<code>parent(X,_).</code>	Unnamed, don't-care value (Ch. 5).
<code>atom</code>	Atom	<code>john, red</code>	A fixed, specific constant (Ch. 2).
<code>'Atom'</code>	Quoted atom	<code>'John Smith'</code>	Allows spaces/uppercase in an atom (Ch. 2).
<code>5, 3.14</code>	Number	<code>age(bob, 25).</code>	Integer or floating-point value (Ch. 2, 10).
<code>f(a,b)</code>	Compound term	<code>point(3,4)</code>	Functor with arguments (Ch. 2, 17).
<code>[]</code>	Empty list	<code>[]</code>	List with zero elements (Ch. 11).
<code>[a,b,c]</code>	List	<code>[1,2,3]</code>	Ordered collection of terms (Ch. 11).
<code>[H T]</code>	Head-tail notation	<code>[H T]=[1,2,3]</code>	Splits first element from the rest (Ch. 11).
<code>%</code>	Line comment	<code>% note</code>	Ignored by Prolog (Ch. 2).
<code>/* */</code>	Block comment	<code>/* note */</code>	Multi-line ignored text (Ch. 2).
<code>,</code>	AND	<code>a, b.</code>	Both goals must succeed (Ch. 2, 9).
<code>;</code>	OR	<code>a ; b.</code>	At least one goal must succeed (Ch. 2, 9).
<code>\+</code>	NOT (negation as failure)	<code>\+ a.</code>	Succeeds if a cannot be proven (Ch. 9, 16).
<code>=</code>	Unification	<code>X = 5.</code>	Makes two terms identical by binding vars (Ch. 7).
<code>is</code>	Arithmetic evaluation	<code>X is 2+3.</code>	Evaluates and binds a numeric result (Ch. 10).
<code>>, <, >=, <=</code>	Numeric comparison	<code>5 > 3.</code>	Compares evaluated numeric values (Ch. 10).
<code>:=, =\=</code>	Arithmetic (in)equality	<code>2+2 := 4.</code>	Compares evaluated results (Ch. 10).
Recursive clause	Recursion	<code>fact(0,1). fact(N,F):-...</code>	Predicate defined in terms of itself (Ch. 12).
<code>!</code>	Cut	<code>max(X,Y,X):-X>=Y,!.</code>	Commits to choices made so far (Ch. 25).
<code>write/1, nl/0</code>	Output	<code>write(hi), nl.</code>	Prints a term / newline (Ch. 18, 19).

Syntax	Meaning	Example	Explanation
<code>read/1</code>	Input	<code>read(X) .</code>	Reads a typed term (Ch. 19).

Appendix B

Common Errors and Fixes

Error	Meaning	Cause	Solution	Example
Unknown procedure	Predicate not defined	Typo in predicate name, or file not loaded	Check spelling; ensure file is loaded with <code>[file]</code> .	<code>studnet(x).</code>
Syntax error	Malformed clause	Missing period, mismatched brackets	Check every clause ends with <code>.;</code> count brackets	<code>likes(a,b)</code> (no period)
Not sufficiently instantiated	Instantiation error	Using <code>is</code> or a builtin with an unbound variable that needs a value	Bind the variable first, or reorder goals	<code>X is Y+1</code> with <code>Y</code> unbound
Type error	Wrong kind of term supplied	Passing an atom where a number was expected	Check argument types match what a predicate needs	<code>X is foo+1</code>
Evaluation error: <code>zero_divisor</code>	Division by zero	An arithmetic expression divides by 0	Guard the divisor with a check before dividing	<code>X is 5/0</code>
Stack overflow / out of memory	Infinite recursion	Missing or unreachable base case	Add or fix the base case; verify recursive case shrinks input	Missing <code>count_down(0)</code> clause
Unexpected false	Query fails when true expected	Typo, missing fact, wrong argument order	Re-check facts/rules; test smaller sub-goals individually	Argument order swapped in <code>parent(bob,tom)</code>
Too many solutions	Query returns more answers than expected	Overlapping clauses, missing inequality constraint	Add <code>X \= Y</code> or restructure clauses	Missing <code>X\=Y</code> in <code>sibling/2</code>
Permission error: modify static procedure	Cannot assert/retract	Predicate not declared dynamic	Add <code>:-dynamic(pred/N).</code> before use	<code>assertz(score(a,1))</code> without declaration
Singleton variable warning	Variable used once	Likely a typo, or intentional use of an anonymous variable	Rename to <code>_</code> if intentional, otherwise fix the typo	<code>foo(X):-bar(Y).</code>

Appendix C

Prolog Practice Problems

This appendix provides 20 beginner, 20 intermediate, and 15 advanced problems. Selected problems include hints. Complete solutions to a representative subset are given in Section C.4 — attempt each problem yourself before checking.

C.1 Beginner Problems (20)

1. Write a fact base of five countries and their capitals; query for the capital of one.
2. Write `square(N, Sq)` computing N squared. *Hint: use `is`.*
3. Write `cube(N, Cube)` computing N cubed.
4. Write `is_positive(N)` that succeeds if N is greater than 0.
5. Write `is_negative(N)`.
6. Write `absolute(N, Abs)` without using the built-in `abs/1`. *Hint: use if-then-else on the sign of N .*
7. Write `swap(A, B, B, A)` that swaps the order of two given terms.
8. Write facts for five students and their favorite colors; query all students who like “blue”.
9. Write `triple_or_more(N)` that succeeds if $N \geq 3$.
10. Write `first_letter_vowel(Atom)` using `atom_chars/2` and `member/2`. *Hint: check the first character against `[a,e,i,o,u]`.*
11. Write `list_of_three(A,B,C,List)` that builds `[A,B,C]`.
12. Write `is_single_digit(N)` that succeeds if N is between 0 and 9.
13. Write `concat_names(First, Last, Full)` using `atomic_list_concat/2`.
14. Write `double_all(List, Doubled)` using recursion.
15. Write `halve_all(List, Halved)`.
16. Write `count_true(List, Count)` that counts how many elements equal the atom `true`.
17. Write a rule `can_vote(Age)` that succeeds if $\text{Age} \geq 18$.
18. Write a small database of animals and their sounds; query the sound a specific animal makes.
19. Write `is_vowel(Char)` for a single character atom.
20. Write `print_greeting(Name)` that prints “Hello, Name!” using `format/2`.

C.2 Intermediate Problems (20)

1. Write `flatten_list(NestedList, Flat)` that flattens one level of nested lists. *Hint: use `append/3` recursively.*
2. Write `remove_first(X, List, Result)` that removes only the first occurrence of X .
3. Write `list_min(List, Min)` without using the built-in `min_list/2`.
4. Write `list_max(List, Max)` without using the built-in `max_list/2`.
5. Write `is_sorted(List)` that checks if a list is already sorted ascending.

6. Write `intersection_custom(List1, List2, Result)` without the built-in `intersection/3`.
7. Write `union_custom(List1, List2, Result)`.
8. Write `rotate_left(List, Rotated)` that moves the first element to the end.
9. Write `zip(List1, List2, Zipped)` that pairs corresponding elements from two equal-length lists.
10. Write `split_at(N, List, Before, After)`.
11. Write `is_anagram(Word1, Word2)` using `msort/2`. *Hint: two words are anagrams if their sorted character lists match.*
12. Write `deepest_nesting(NestedList, Depth)` computing how deeply nested a list is.
13. Write `run_length_encode(List, Encoded)` performing simple run-length encoding, e.g. `[a,a,b] -> [2-a,1-b]`.
14. Write `tree_sum(Tree, Sum)` for a binary tree represented as `tree(Value,Left,Right)` or `leaf`.
15. Write `tree_depth(Tree, Depth)` for the same tree representation.
16. Write `binary_search(List, Target)` assuming `List` is sorted. *Hint: split the list in half using `length/2` and `append/3`.*
17. Write a small graph and a predicate `connected(X, Y)` that checks reachability, avoiding infinite loops on cycles by tracking visited nodes.
18. Write `sum_digits(N, Sum)` computing the sum of the digits of a non-negative integer `N`.
19. Write `is_prime(N)` checking whether `N` is a prime number.
20. Write `gcd_custom(A, B, GCD)` computing the greatest common divisor recursively.

C.3 Advanced Problems (15)

1. Solve the 4-Queens problem using `CLP(FD)` and list all solutions.
2. Implement a Sudoku solver for a partially filled 4x4 grid using `CLP(FD)`.
3. Write a cryptarithm solver for `TWO + TWO = FOUR`.
4. Write a simple job-scheduling program: three jobs, each with a duration, must be scheduled without overlapping on one machine, using `CLP(FD)`.
5. Extend the DCG grammar from Chapter 27 to support simple negation, e.g. “the cat does not see the dog”.
6. Write a DCG that parses simple arithmetic expressions with `+` and `*` and evaluates them.
7. Implement a small forward-chaining rule engine using `assert/1` to add newly derived facts until no more can be derived.
8. Write a predicate that finds all Hamiltonian paths in a small graph (a path visiting every node exactly once).
9. Implement the N-Queens problem using plain generate-and-test (Chapter 21) instead of `CLP(FD)`, and compare its performance to the `CLP(FD)` version for `N=8`.
10. Write an expert-system style predicate that explains its reasoning (prints which rules fired) when reaching a conclusion.
11. Extend the University Academic Advisor (Chapter 30) with a predicate that produces a full multi-semester course plan given a maximum number of credits per semester.
12. Write a predicate that checks if a given assignment of colors to a map’s regions is valid (map coloring problem), then use `CLP(FD)` to find a valid coloring with 4 colors.
13. Implement a simple magic square checker for a 3x3 grid of numbers.
14. Write a solver for the classic “Zebra Puzzle” (or a simplified version with 3 houses instead of 5) using `CLP(FD)` or plain generate-and-test.
15. Extend the Sudoku solver from Chapter 26 to read a puzzle from a list of 81 numbers (using 0 for blanks) and print the solved grid neatly using `format/2`.

C.4 Selected Complete Solutions

Beginner #6 (absolute value):

```
absolute(N, N) :-N >= 0, !.
absolute(N, Abs) :-N < 0, Abs is -N.
```

Intermediate #3 (list minimum):

```
list_min([X], X).
list_min([H|T], Min) :-
    list_min(T, TailMin),
    ( H =< TailMin ->Min = H ;Min = TailMin ).
```

Intermediate #18 (sum of digits):

```
sum_digits(N, Sum) :-
    N < 10, !, Sum = N.
sum_digits(N, Sum) :-
    N >= 10,
    Last is N mod 10,
    Rest is N // 10,
    sum_digits(Rest, RestSum),
    Sum is Last + RestSum.
```

Intermediate #19 (primality test):

```
is_prime(2) :-!.
is_prime(N) :-
    N > 2,
    N mod 2 =\= 0,
    \+ has_odd_divisor(N, 3).

has_odd_divisor(N, D) :-
    D * D =< N,
    ( N mod D =:= 0
    -> true
    ; D2 is D + 2,
      has_odd_divisor(N, D2)
    ).
```

Advanced #1 (4-Queens with CLP(FD)):

```
:- use_module(library(clpfd)).

queens4(Qs) :-
    Qs = [Q1,Q2,Q3,Q4],
    Qs ins 1..4,
    all_distinct(Qs),
    abs(Q1-Q2) #\= 1, abs(Q1-Q3) #\= 2, abs(Q1-Q4) #\= 3,
    abs(Q2-Q3) #\= 1, abs(Q2-Q4) #\= 2,
    abs(Q3-Q4) #\= 1,
    label(Qs).
```

```
?- queens4(Qs).
Qs = [2, 4, 1, 3] ;
Qs = [3, 1, 4, 2].
```


Appendix D

Viva / Interview Questions

The following 50 questions cover the full range of topics in this book and are suitable for viva voce examinations, interviews, or self-testing.

1. What does the name “Prolog” stand for, and what does it mean?
2. What is the difference between declarative and procedural programming?
3. What is a clause in Prolog?
4. Define a fact and give an example.
5. Define a rule and explain the meaning of `:-`.
6. What is the difference between a fact and a rule?
7. What is a predicate, and how is its arity determined?
8. What is a term in Prolog? Name the four kinds of terms.
9. What distinguishes an atom from a variable syntactically?
10. What is the anonymous variable, and when should it be used?
11. Explain variable scope in Prolog.
12. Can the same variable name in two different clauses refer to the same variable? Why or why not?
13. What does it mean for a variable to be “bound”?
14. What is a query, and how does it differ from a fact?
15. What does a comma mean inside a rule body? What does a semicolon mean?
16. What is unification, and how does it differ from assignment in languages like Python?
17. Why does `X = 2 + 3` not bind `X` to 5?
18. What happens when two compound terms with different arities are unified?
19. What is pattern matching in Prolog, and how is it related to unification?
20. What does `[H|T]` represent, and what does it match?
21. What is the difference between `=` and `is`?
22. Why does `2 + 2 = 4` fail while `2 + 2 == 4` succeeds?
23. List the arithmetic comparison operators available in Prolog.
24. What is a list in Prolog, and how is it represented internally?
25. What does the built-in `append/3` predicate do, and can it run “backwards”?
26. What is the difference between `sort/2` and `msort/2`?
27. Define recursion, and explain the role of the base case.
28. Why does Prolog rely on recursion instead of loops?
29. Trace through `factorial(3, F)` step by step.
30. What is backtracking, and why is it central to Prolog’s execution model?
31. What is a choice point?
32. What is the difference between typing `;` at the toplevel and writing `;` in a rule body?

33. Explain the concept of resolution in Prolog.
34. What is depth-first search, and how does Prolog use it?
35. What does `trace/0` do, and what do `Call`, `Exit`, `Fail`, and `Redo` mean?
36. What is negation as failure, and how is it different from classical logical negation?
37. Why can negation as failure give incorrect results with unbound variables?
38. Explain the closed-world assumption.
39. What is a compound term, and how does it differ from a list?
40. Name five built-in predicates and explain what each does.
41. What is the difference between `write/1` and `writeln/1`?
42. What does the cut (!) do, and what is the difference between a green cut and a red cut?
43. What is the purpose of `assert/1` and `retract/1`?
44. Explain the difference between `findall/3`, `bagof/3`, and `setof/3`.
45. What is an accumulator, and why is it useful for recursion?
46. What does it mean for a predicate to be tail recursive?
47. What is a Definite Clause Grammar, and what operator does it use?
48. Explain what `phrase/2` does.
49. What is Constraint Logic Programming, and how does it differ from ordinary arithmetic in Prolog?
50. What is the purpose of `label/1` in CLP(FD)?
51. Why is Prolog historically significant in Artificial Intelligence, and how does it compare to modern machine learning approaches?

Appendix E

Quick Reference: Prolog in One Page

Prolog in One Page

Facts & Rules

```
parent(tom, bob).           % fact
happy(X) :-likes(X, pizza). % rule: "if"
```

Terms

Atom: john Variable: X Number: 5, 3.14 Compound: point(1,2) Anonymous: _

Logical Operators

AND: , OR: ; NOT: \+ If-then-else: (C -> T ; E)

Unification vs Arithmetic

```
X = 5.           % unification: bind X to 5
X is 2+3.        % evaluate arithmetic: X = 5
2+2 == 4.        % arithmetic equality: true
2+2 = 4.         % structural unification: false
```

Comparisons

> < >= <= := \= (all evaluate both sides first)

Lists

```
[]                % empty list
[1,2,3]           % list literal
[H|T]             % head/tail split
[A,B|Rest]        % first two + rest
```

Common built-ins: member/2, append/3, length/2, reverse/2, sort/2, nth0/3

Recursion Template

```
pred([], BaseCase).
pred([H|T], Result) :-
    pred(T, PartialResult),
    combine(H, PartialResult, Result).
```

Backtracking

Prolog tries the first matching clause; on failure (or when asked for more via ;), it backtracks to the last choice point and tries the next alternative.

Negation as Failure

```
\+ Goal % succeeds if Goal cannot be proven
        % only reliable if Goal is ground!
```

Cut

```
max(X,Y,X) :-X >= Y, !.
max(_,Y,Y).
```

Commits to choices made so far; prevents backtracking past that point.

Aggregation

```
findall(X, Goal, List). % all solutions (list, may be empty)
bagof(X, Goal, List).  % all solutions (fails if none)
setof(X, Goal, List).  % sorted, unique solutions
```

I/O

```
write(X). writeln(X). nl. read(X).
format("Name: ~w, Age: ~d~n", [Name, Age]).
```

Dynamic Database

```
:- dynamic(fact/2).
assertz(fact(a,b)).
retract(fact(a,b)).
retractall(fact(_,_)).
```

Constraint Logic Programming (CLP(FD))

```
:- use_module(library(clpfd)).
Vars ins 1..N, all_distinct(Vars),
X #= Y + 1, X #\= Y, label(Vars).
```

Definite Clause Grammar

```
sentence -> noun_phrase, verb_phrase.
?- phrase(sentence, [the, cat, sees, a, dog]).
```